claude-windows / b42b585b-e38f-48ef-8573-5cfc6da69801

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801.jsonl`
- SHA-256: `b7897f1484e267b963b495e2249e10d88eaeb9b63756735e0d6e93a95d83c46b`
- Source modified: `2026-07-02T16:36:59+00:00`
- Imported at: `2026-07-05T16:48:18+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `b42b585b-e38f-48ef-8573-5cfc6da69801`

Transcript

1. user (2026-06-22T18:28:37.740Z)

In backend/compute/cloud_run.py, GoogleCloudRunJobClient.run_job calls `client.job_path(project, region, job_name)` where `project` is `Optional[str]` (from `GOOGLE_CLOUD_PROJECT` env, can be None) but `JobsClient.job_path` expects `str`. mypy flags this (only when google-cloud-run is installed, so CI ? which doesn't install it ? stays green, but it's a real latent bug: a None project would fail the real Cloud Run dispatch). Fix by validating project is set (raise a clear error if GOOGLE_CLOUD_PROJECT is unset) before calling job_path, or narrowing the type. Add a small test for the missing-project case. Keep it minimal; run ruff + `mypy backend` (with google-cloud-run installed) + the compute tests.

2. assistant (2026-06-22T18:28:52.875Z)

I'll start by understanding the current state and reading the target file. Let me gather context in parallel.

3. user (2026-06-22T18:28:53.840Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4 GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7): the
10           cold-start is absorbed by the async poll.
11        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` + native detector)
15      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just computes.
16
17      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole shim is unit-tested
18      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
19      owner-verified on the M7 real run.
```

```
    """

    from __future__ import annotations

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, List, Optional

    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
    from backend.storage import fetch, parse_uri
    from backend.storage import get_storage

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
        a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""

        def run_job(self, job_name: str, argv: List[str], *, region: str,
                    project: Optional[str] = None) -> str:
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
```

```
            name = client.job_path(project, region, job_name)
            overrides = run_v2.RunJobRequest.Overrides(

container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
            )
            op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
            return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
                     project: Optional[str] = None, storage_config: Optional[dict] = None,
                     policy: ExecutionPolicy = DEFAULT_POLICY,
                     token: Optional[str] = None,
                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
            )

        def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
                               "--done-uri", done_uri]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
                argv += ["--set", kv]

            execution = self._client.run_job(self._job_name, argv, region=self._region,
                                             project=self._project)
            LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                     self._job_name, execution, done_uri)

            marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
                                label="cloud-run-infer", logger=LOG)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
                            "video_id) ? treating as failure", self._job_name)
                rc = rc or 1
```

```
129             outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130             report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131             LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132             return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:
136             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137             ref = parse_uri(done_uri)
138             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139                 return None
140             return self._read_json(done_uri) or {}  # empty dict still terminates the poll
(present == done)
141
142         def _read_json(self, uri: str) -> dict:
143             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146             try:
147                 ref = parse_uri(uri)
148                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149                 local = fetch(uri, dst, config=self._storage_config)
150                 with open(local, "r", encoding="utf-8") as f:
151                     data: Any = json.load(f)
152                 return data if isinstance(data, dict) else {}
153             except Exception:
154                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155                 return {}
156
```

4. user (2026-06-22T18:28:55.566Z)

```
backend\compute\__init__.py
backend\compute\cloud_run.py
backend\compute\base.py
backend\compute\__pycache__\__init__.cpython-313.pyc
backend\compute\__pycache__\base.cpython-313.pyc
backend\compute\__pycache__\cloud_run.cpython-313.pyc
```

5. user (2026-06-22T18:28:55.700Z)

```
tests\test_compute_backend.py
tests\__pycache__\test_compute_backend.cpython-313-pytest-9.0.3.pyc
```

6. user (2026-06-22T18:28:56.609Z)

```
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---BRANCH---
---HEAD---
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
```

7. user (2026-06-22T18:29:02.154Z)

```
1       """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
```

```
    All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that simulates the
    worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the dispatch?bucket-poll
    ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only ``compute_target=cloud_run``
    selects a backend; everything else returns ``None`` (= run the worker's existing in-process path).
    """
    import json
    import os
    import tempfile

    from backend.compute import ComputeJobSpec, get_compute_backend
    from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
    from backend.config import AppConfig
    from backend.infrastructure.execution_policy import ExecutionPolicy

    # A fast, no-wait policy (the fake client writes the marker synchronously, so the first poll hits).
    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)


    class _FakeRunClient(CloudRunJobClient):
        """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the local bucket
        exactly where the real worker would, so CloudRunCompute's bucket-poll resolves immediately."""

        def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
            self.calls: list[dict] = []
            self.video_id, self.segment_count, self.returncode = video_id, segment_count, returncode

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append({"job": job_name, "argv": list(argv), "region": region, "project": project})
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            outputs = os.path.join(out_uri, "outputs", self.video_id)
            os.makedirs(outputs, exist_ok=True)
            with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "segment_count": self.segment_count,
                           "status": "success"}, f)
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "returncode": self.returncode,
                           "segment_count": self.segment_count, "status": "success"}, f)
            return "exec-123"


    class _NoopRunClient(CloudRunJobClient):
        """Records the dispatch but does NO filesystem writes ? for tests that fake the storage layer
        directly (a gs:// out_uri has no local dir to makedirs)."""

        def __init__(self):
            self.calls: list[dict] = []

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append({"job": job_name, "argv": list(argv), "region": region, "project": project})
            return "exec"


```

```
    # ------------------------------------------------------------------ factory (default-OFF)

    def _cfg(compute_target="", storage_backend="local"):
        return AppConfig.model_validate({
            "deployment": {"compute_target": compute_target},
            "storage": {"backend": storage_backend},
        })


    def test_factory_is_none_for_local_targets():
        """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process path)."""
        assert get_compute_backend(_cfg("")) is None
        assert get_compute_backend(_cfg("local_gpu")) is None
        assert get_compute_backend(_cfg("cpu_mock")) is None


    def test_factory_returns_cloud_run_for_cloud_target():
        backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
        assert isinstance(backend, CloudRunCompute)


    # ------------------------------------------------------------ CloudRunCompute dispatch + poll

    def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
        client = _FakeRunClient(video_id="abc123", segment_count=3)
        backend = CloudRunCompute(run_client=client, job_name="rally-worker", region="asia-south1",
                                  project="proj", storage_config={}, policy=_FAST, token="tok")
        out = str(tmp_path / "bucket")
        result = backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"))

        assert result.returncode == 0
        assert result.video_id == "abc123"
        assert result.outputs_uri == f"{out}/outputs/abc123/"
        assert result.report.get("segment_count") == 3
        # exactly one Cloud Run Job execution was triggered, in the right region/project
        assert len(client.calls) == 1
        assert client.calls[0]["job"] == "rally-worker" and client.calls[0]["region"] == "asia-south1"


    def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
        """The dispatched container must run INLINE (never re-dispatch) ? compute_target=local_gpu +
        native detector appended AFTER the user overrides (so they win); --done-uri always present."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST, token="tok")
        backend.run_infer(ComputeJobSpec(
            video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"), segmenter="wasb_hybrid",
            config_overrides=("sport=tennis",)))

        argv = client.calls[0]["argv"]
        assert argv[0] == "infer"
        assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        # the container-inline overrides come last (win) and the user override is carried through
        joined = " ".join(argv)
```

```
        assert "deployment.compute_target=local_gpu" in joined
        assert "indexing.detector_impl=native" in joined
        assert "sport=tennis" in joined
        # user override precedes the inline-forcing overrides in --set order
        assert joined.index("sport=tennis") <
joined.index("deployment.compute_target=local_gpu")


    def test_unique_token_per_call_when_not_injected(tmp_path):
        """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
        client = _FakeRunClient()
        backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path /
"bk")))
        done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
        done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
        assert done1 != done2


    def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
        """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
        pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
        identity passthrough). Mimic a cloud backend that rejects dst=None."""
        from backend.compute import cloud_run as cr

        captured = {}

        def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
            captured["dst"] = dst
            if dst is None:
                raise ValueError("cloud backend requires a dst path")  # mirror
GCSStorage/S3Storage
            with open(dst, "w", encoding="utf-8") as f:
                f.write('{"video_id": "v", "returncode": 0}')
            return dst

        monkeypatch.setattr(cr, "fetch", fake_fetch)
        backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
        data = backend._read_json("gs://b/x.json")
        assert captured["dst"] is not None  # the fix: a real dst is always passed (no
silent {})
        assert data == {"video_id": "v", "returncode": 0}


    def test_cloud_run_polls_until_marker_appears(monkeypatch):
        """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
        polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
        never exercises this; here exists() is False twice then True."""
        from backend.compute import cloud_run as cr

        exists_calls = {"n": 0}

        class _FakeStore:
            def exists(self, ref):
                exists_calls["n"] += 1
                return exists_calls["n"] >= 3  # absent for polls 1-2, present on poll 3

        monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
```

```
167
168         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
169             if dst is None:
170                 dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
171             with open(dst, "w", encoding="utf-8") as f:
172                 f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
173             return dst
174
175         monkeypatch.setattr(cr, "fetch", fake_fetch)
176         backend = cr.CloudRunCompute(
177             run_client=_NoopRunClient(), job_name="j", region="r", token="t",
178             policy=ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0,
max_wait_sec=5.0))
179         result = backend.run_infer(ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b"))
180         assert result.video_id == "vlate"
181         assert exists_calls["n"] >= 3  # >=1 absent poll occurred before the marker appeared
(loop ran)
182
183
184     # ---------------------------------------------------- worker wiring: dispatch branch +
done-marker
185
186     def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
187         """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
process run)."""
188         from backend.compute.base import ComputeResult
189         from backend.worker import __main__ as wmain
190
191         seen = {}
192
193         class _FakeBackend:
194             def run_infer(self, spec):
195                 seen["spec"] = spec
196                 return ComputeResult(returncode=0, video_id="vidX",
outputs_uri="gs://b/outputs/vidX/")
197
198         monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
199         cfg = tmp_path / "c.json"
200         cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"gcs"}}')
201         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
202             "infer", "--video-uri", "gs://b/videos/clip.mp4", "--out-uri", "gs://b",
203             "--config", str(cfg), "--segmenter", "wasb_hybrid"]))
204         assert rc == 0
205         assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
206         assert seen["spec"].segmenter == "wasb_hybrid"
207
208
209     def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
210         """The container side: cmd_infer with --done-uri writes a completion marker
(video_id + rc) so a
211         remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
AI."""
212         from backend.worker import __main__ as wmain
213
214         class _OK:
215             passed = True
216             validator_name = "fake"
217             message = "ok"
218
219             def model_dump(self):
220                 return {"validator_name": "fake", "passed": True, "message": "ok"}
221
222         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
```

```
                            lambda path, cfg: ([_OK()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)

monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
                            lambda p: 30.0)

        cfg = tmp_path / "c.json"
        cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
                                   "output": {"output_dir": str(tmp_path / "o")}}))
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        done = str(out / "_dispatch" / "tok.done")

        rc = wmain.cmd_infer(wmain.build_parser().parse_args([
            "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
            "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid", "--done-uri",
done]))
        assert rc == 0
        marker = json.loads(open(done, encoding="utf-8").read())
        assert marker["returncode"] == 0 and marker["status"] == "success"
        assert marker["video_id"] and marker["segment_count"] >= 1


    def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
        """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
        from backend.worker import __main__ as wmain

        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
                return {"validator_name": "fake", "passed": True, "message": "ok"}

        monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                            lambda path, cfg: ([_OK()], {}))
        monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)

monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
                            lambda p: 30.0)
        cfg = tmp_path / "c.json"
        cfg.write_text(json.dumps({"indexing": {"detector_impl": "stub", "skip_ai_handoff":
True},
                                   "output": {"output_dir": str(tmp_path / "o")}}))
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO")
        out = tmp_path / "bucket"
        rc = wmain.cmd_infer(wmain.build_parser().parse_args([
            "infer", "--video-uri", str(video), "--out-uri", str(out), "--config", str(cfg),
            "--scratch", str(tmp_path / "s"), "--segmenter", "wasb_hybrid"]))
        assert rc == 0
        assert not (out / "_dispatch").exists()  # no marker dir when --done-uri is omitted


    # ------------------------------------------------------- Dockerfile structural guard
(no build)

    def test_dockerfile_has_required_directives():
        """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
structurally:
        the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
```

```
worker entrypoint."""
280         from pathlib import Path
281
282         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
283         assert "FROM nvidia/cuda" in text                       # GPU base (CUDA)
284         assert "ffmpeg" in text                                  # stitch/decode
285         assert "fonts-dejavu" in text                            # the watermark filter font
286         assert "torch==1.11.0+cu113" in text                    # the verified WASB stack
287         assert "WASB-SBDT" in text                               # the shuttle detector repo
288         assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
289         assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
290
```

8. assistant (2026-06-22T18:29:08.854Z)

Let me look at the factory to understand how `project` flows from `GOOGLE_CLOUD_PROJECT`.

9. user (2026-06-22T18:29:09.432Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
```

```
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

10. assistant (2026-06-22T18:30:01.233Z)

```
I now fully understand the flow:

- `get_compute_backend` ([backend/compute/__init__.py:56](backend/compute/__init__.py)) reads
`project = os.environ.get("GOOGLE_CLOUD_PROJECT")` ? `None` if unset.
- That flows to `CloudRunCompute._project` ? `run_job(project=...)` ? `client.job_path(project,
region, job_name)` ([backend/compute/cloud_run.py:74](backend/compute/cloud_run.py)), where
`job_path` requires `str`.

The fix: guard `project` at the top of `run_job` (before the lazy SDK import, so the guard is
testable in CI where `google-cloud-run` is absent). This both narrows `Optional[str]` ? `str`
for mypy and makes a misconfigured dispatch fail loudly.

Let me check the lint/type config and whether the SDK is installed, then make the edit.
```

11. user (2026-06-22T18:30:03.991Z)

```
=== ruff/mypy config (pyproject) ===
```

PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).

```
#
```

Build backend: hatchling. `backend/` is an intentional NAMESPACE package (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's auto-detection would otherwise miss the un-__init__'d `backend` root.

```
#
```

torch/torchvision are deliberately NOT listed in [project.dependencies]: their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url), which PEP 621 cannot express. Install them separately ? see [project.optional-depend `gpu` (documentation only) and the README. CI installs CPU torch on its own line.

```
[build-system]
```

```
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
```

Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).

```
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0,<0.30.0",
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
]

[project.optional-dependencies]
```

Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).

```
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]
```

M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_a

`jwt` is lazy-imported, so the base runtime works without this; install it for hosted/mul

```
auth = ["PyJWT[crypto]>=2.8"]
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Pinned to its git tag per BACKLOG. The private repo is fetched over SSH; if you lack access, simply omit this group ? reporting tests skip via pytest.importorskip. For local dev against a sibling checkout: pip install -e ../sports-obsreport --no-build-isolation

```
reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]
```

`gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot express, so they are NOT declarable here. Install them out-of-band instead:

 # CUDA 12.4 box:
pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org
 # CPU-only box (also what CI uses):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

```
gpu = []
```

Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs
these. Cloud SDKs are imported lazily, so `import backend.storage` works without the
install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2
DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storag

```
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
```

`indexer --server`, `indexer --help`, etc. -> backend/main.py::main()

```
indexer = "backend.main:main"

[tool.hatch.metadata]
```

The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct reference*); hatchling requires opting in to allow those in metadata.

```
allow-direct-references = true

[tool.hatch.build.targets.wheel]
```

`backend` is a namespace package (no __init__.py); list both import roots explicitly so an editable/wheel install exposes `import backend` and `import training` regardless of cwd.

```
packages = ["backend", "training"]
=== setup.cfg ===
=== mypy.ini ===
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set only in main()); properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).

```
[mypy-backend.main]
ignore_errors = True
```

Job-worker drain/wake loop extracted from backend.main (same Optional module-glo

debt: resolves db_instance/global_config THROUGH backend.main at call-time). Share backend.main's quarantine until the deferred app-factory/lifespan refactor lands.

```
[mypy-backend.api.job_worker]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

Legacy demo segmenter (fabricated metadata; coverage-omitted too).

```
[mypy-backend.pipeline.segmenters.motion]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
=== ruff.toml ===
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

```
#
```

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.

```
target-version = "py38"
```

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B"]

[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config,

sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]
```

12. user (2026-06-22T18:30:06.680Z)

```
run_v2 OK C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2\__init__.py
---
Traceback (most recent call last):
  File "<string>", line 1, in <module>
---
ruff 0.15.16
```

13. assistant (2026-06-22T18:30:26.466Z)

`google-cloud-run` (run_v2) is already installed and `cloud_run.py` is NOT in the mypy quarantine list ? so mypy checks it. Let me confirm the mypy invocation CI uses, verify mypy runs, and **reproduce the error before fixing** (baseline).

14. user (2026-06-22T18:30:28.142Z)

```
mypy 2.1.0 (compiled: yes)
=== CI mypy/ruff invocation ===
16:  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17:  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
28:      - name: Install ruff
29:        run: python -m pip install ruff
31:        run: ruff check . --output-format github
48:  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
50:  # up. Typed core deps are installed so mypy sees their real signatures
60:      - name: Install mypy + typed core deps
61:        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
63:        run: mypy backend training
102:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
```

15. assistant (2026-06-22T18:30:37.132Z)

mypy 2.1.0 is available, CI runs `mypy backend training` (and notably does **not** install `google-cloud-run`, confirming why CI stays green). Let me reproduce the latent error locally where the SDK *is* installed:

16. user (2026-06-22T18:30:40.604Z)

```
Success: no issues found in 1 source file
```

17. assistant (2026-06-22T18:30:48.074Z)

Single-file check resolves imports differently. Let me run the exact CI invocation `mypy backend` and filter for the relevant line:

18. user (2026-06-22T18:30:54.988Z)

```
=== full tail ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
```

```
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 133 source files
```

19. assistant (2026-06-22T18:31:04.138Z)

mypy reports success even with the SDK installed. Let me investigate the actual `job_path` signature and whether the package ships type info (`py.typed`):

20. user (2026-06-22T18:31:08.817Z)

```
run_v2 version: 0.16.0
job_path sig: (project: str, location: str, job: str) -> str
py.typed at C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud\run_v2 => True
py.typed at C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\cloud => False
----- job_path source -----
    @staticmethod
    def job_path(
        project: str,
        location: str,
        job: str,
    ) -> str:
        """Returns a fully-qualified job string."""
        return "projects/{project}/locations/{location}/jobs/{job}".format(
            project=project,
            location=location,
            job=job,
        )
```

21. assistant (2026-06-22T18:31:30.919Z)

`job_path(project: str, ...)` is strictly typed and `run_v2` ships `py.typed`. Yet `mypy backend` passes. The reason is the `# type: ignore[attr-defined]` on the import line ? let me confirm whether that's masking the error, since I want to be sure the fix actually resolves what mypy sees. Let me probe with `reveal_type`:

22. user (2026-06-22T18:31:36.369Z)

```
scratch\_mypy_probe.py:5: note: Revealed type is "types.ModuleType"
scratch\_mypy_probe.py:7: note: Revealed type is "def (project: str, location: str, job: str) ->
str"
scratch\_mypy_probe.py:8: error: Argument 1 to "job_path" of "JobsClient" has incompatible type
"str | None"; expected "str"  [arg-type]
scratch\_mypy_probe.py:13: note: Revealed type is "types.ModuleType"
scratch\_mypy_probe.py:15: note: Revealed type is "def (project: str, location: str, job: str)
-> str"
scratch\_mypy_probe.py:16: error: Argument 1 to "job_path" of "JobsClient" has incompatible type
"str | None"; expected "str"  [arg-type]
Found 2 errors in 1 file (checked 1 source file)
```

23. assistant (2026-06-22T18:32:00.668Z)

mypy **does** flag it: `Argument 1 to "job_path"... incompatible type "str | None"; expected "str"`. But the broad `mypy backend` run passed ? I need to reconcile that before I trust my verification. Let me re-run `mypy backend` freshly (no cache) and target the module directly:

24. user (2026-06-22T18:32:41.754Z)

```
=== mypy backend --no-incremental, grep cloud_run/arg-type ===
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient" has
incompatible type "str | None"; expected "str"  [arg-type]
exit: done
=== mypy -m backend.compute.cloud_run ===
```

backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient" has
incompatible type "str | None"; expected "str" [arg-type]
Found 1 error in 1 file (checked 1 source file)

25. assistant (2026-06-22T18:33:01.555Z)

Confirmed ? the earlier pass was a stale `.mypy_cache` artifact. With `--no-incremental` the
error reproduces at exactly **line 74** as described:

```
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path"... incompatible type "str |
None"; expected "str"  [arg-type]
```

Now I'll make the fix. I'll guard `project` at the top of `run_job` ? **before** the lazy SDK
import, so the guard is exercisable in CI (where `google-cloud-run` is absent) and narrows
`Optional[str]` ? `str` for mypy.

26. user (2026-06-22T18:33:06.101Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-22T18:33:09.507Z)

Now the test for the missing-project case. I'll add it to the compute test file, near the other
dispatch tests:

28. user (2026-06-22T18:33:10.684Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

29. user (2026-06-22T18:33:18.253Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

30. assistant (2026-06-22T18:33:22.604Z)

Now clean up the probe file and run the full verification ? ruff, mypy (no-incremental, SDK
installed), and the compute tests:

31. user (2026-06-22T18:34:05.384Z)

```
=== RUFF (repo-wide, CI lane) ===
help: Rewrite `bold` as a `def`

E731 Do not assign a `lambda` expression, use a `def`
  --> artifacts\landing\make_og.py:24:1
   |
23 | bold = lambda s: font("segoeuib.ttf", s)
24 | reg = lambda s: font("segoeui.ttf", s)
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
25 |
26 | M = 80  # margin
   |
help: Rewrite `reg` as a `def`

Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
=== MYPY backend (no-incremental, SDK installed) ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
```

```
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 136 source files
=== PYTEST compute backend ===
............                                                             [100%]
12 passed in 5.32s
```

32. assistant (2026-06-22T18:34:17.505Z)

mypy now reports **`Success: no issues found in 136 source files`** (the line-74 `arg-type` error is gone), and all **12 compute tests pass** (including the new `test_real_client_rejects_missing_project`).

The 2 ruff errors are in `artifacts/landing/make_og.py` ? a pre-existing, untracked file (`?? artifacts/` in git status) unrelated to my change. Let me confirm my two touched files are ruff-clean:

33. user (2026-06-22T18:34:19.868Z)

```
=== ruff on touched files ===
All checks passed!
ALL CLEAN
=== confirm artifacts is untracked (not my change) ===
 M backend/compute/cloud_run.py
 M tests/test_compute_backend.py
=== diff stat of my change ===
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
 backend/compute/cloud_run.py  |  8 ++++++++
 tests/test_compute_backend.py | 17 +++++++++++++++++
 2 files changed, 25 insertions(+)
```

34. assistant (2026-06-22T18:34:41.533Z)

Both touched files are ruff-clean (the 2 ruff errors are in the untracked `artifacts/landing/make_og.py`, unrelated to this change). The diff is scoped to exactly the two intended files: +8 in the source, +17 in tests.

Here's the final source change:

35. user (2026-06-22T18:34:42.227Z)

```
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
67              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
68              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
69              if not project:
70                  raise RuntimeError(
71                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
72                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
73                  )
74              try:
```

```
75                  from google.cloud import run_v2  # type: ignore[attr-defined]
76              except Exception as exc:  # pragma: no cover - real SDK not present in CI
77                  raise RuntimeError(
78                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
79                      "control plane (it is intentionally NOT a CI/test dependency)."
80                  ) from exc
81              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
82              name = client.job_path(project, region, job_name)
83              overrides = run_v2.RunJobRequest.Overrides(
84
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
85              )
```

36. assistant (2026-06-22T18:34:55.998Z)

Done. Here's what I did and what I verified.

The bug, confirmed

With `google-cloud-run` installed, mypy flagged exactly the line described:

```
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient"
  has incompatible type "str | None"; expected "str"  [arg-type]
```

`job_path(project: str, location: str, job: str)` is strictly typed (the package ships `py.typed`), and `project` arrives as `Optional[str]` from `os.environ.get("GOOGLE_CLOUD_PROJECT")` in [get_compute_backend](backend/compute/__init__.py:56). CI stays green only because CI doesn't install the SDK (mypy then sees `run_v2` as `Any`). It's a real latent bug ? a `None`/unset project builds a bogus `projects/None/locations/...` path on the real dispatch.

> Note: my first `mypy backend` run passed due to a stale `.mypy_cache`. `--no-incremental` reproduced it ? worth flagging since incremental cache can mask this class of error.

The fix

A guard at the top of [`GoogleCloudRunJobClient.run_job`](backend/compute/cloud_run.py:69), placed **before** the lazy `google-cloud-run` import. This:
- **Narrows** `Optional[str]` ? `str` so `job_path(project, ?)` type-checks.
- **Fails loudly** with a clear message naming `GOOGLE_CLOUD_PROJECT` instead of dispatching a malformed path (covers both `None` and `""`).
- Is **testable in CI** ? the guard runs before the SDK import, so the missing-project path is exercisable even though the SDK isn't a test dependency.

Added [`test_real_client_rejects_missing_project`](tests/test_compute_backend.py:81), which asserts a `RuntimeError` matching `GOOGLE_CLOUD_PROJECT` for both `None` and `""`.

Verification (all green)

Check	Command	Result
Types	`mypy backend training --no-incremental` (SDK installed)	**Success, no issues** (136 files); line-74 error gone
Lint	`ruff check backend/compute/cloud_run.py tests/test_compute_backend.py`	**All checks passed**
Tests	`pytest tests/test_compute_backend.py`	**12 passed** (was 11 + my new one)

The 2 ruff errors in the repo-wide run are in `artifacts/landing/make_og.py` ? an untracked file (`?? artifacts/`) pre-existing and unrelated to this change.

Diff is +25 lines across the two intended files only. I did **not** commit or open a PR (you didn't ask, and per project convention). Heads-up: the working tree is on a **detached HEAD** ? if you want this as a PR, I'd cut a branch from `origin/master` and cherry-pick these two files in.

37. user (2026-06-22T18:44:40.729Z)

please make the PRs ensure the tests coverage improve and all CI is green and merge it!

38. assistant (2026-06-22T18:45:01.876Z)

On it. This is one work item ? one PR, and per our working agreement I can merge my own verified-green engineering-fix PRs. Let me set up safely: confirm the remote/auth, check where `origin/master` is, and find the coverage floor + full-suite command.

39. user (2026-06-22T18:45:06.724Z)

```
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
=== remote ===
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
=== fetch ===
=== HEAD vs origin/master ===
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional
O1/O2 (#323)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
```

40. user (2026-06-22T18:45:06.989Z)

```
=== pytest.ini ===
[pytest]
```

When `pytest` is invoked without an explicit path, only collect from tests/. scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

```
testpaths = tests
=== cov-fail-under / cov refs in CI ===
78:        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
80:        run: python -m pytest tests/test_import_smoke.py -v
102:        # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
105:      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
112:        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-
fail-under=70
133:        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
140:          python -m pytest tests/test_golden_fixtures.py tests/test_golden_real_fixtures.py
-q
=== run_all_tests.py head ===
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage

Anything after the script name is passed straight through to pytest.

Notes
```

```
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
```

41. assistant (2026-06-22T18:45:25.974Z)

Key findings:
- **`origin/master` has moved ahead**: my detached HEAD is at `9b37269` (#323) but `origin/master` is at `d92cc10` (#341) ? and #341 is *serving*-related, which could touch `cloud_run.py`. I must branch from `origin/master` and re-apply on top of the real base.
- **Coverage floor is `--cov-fail-under=70`**; full suite = `python run_all_tests.py`.

Let me back up my working changes as a patch, then check whether #341 touched my two files:

42. user (2026-06-22T18:45:29.312Z)

```
warning: in the working copy of 'backend/compute/cloud_run.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_compute_backend.py', LF will be replaced by CRLF the
next time Git touches it
patch bytes:
2639
=== did #323..#341 touch my files? ===
 backend/compute/cloud_run.py  |  3 +-
 tests/test_compute_backend.py | 80 ++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 82 insertions(+), 1 deletion(-)
=== (empty above = unchanged) ===
=== full file list changed #323..origin/master ===
 backend/infrastructure/database.py       |  60 +++-
 backend/main.py                          | 365 ++++++++++++++++++---
 backend/storage/__init__.py              |  10 +-
 backend/storage/gdrive_api.py            | 178 ++++++++++
 backend/storage/ref.py                   |   7 +-
 backend/ui/THIRD_PARTY_NOTICES.md        |   2 +-
 backend/ui/assets/index-BcE6YOgS.css     |   1 -
 backend/ui/assets/index-GSXqL6iY.js      |  60 ----
 backend/ui/assets/index-Jp9DK5o0.css     |   1 +
 backend/ui/assets/index-ukuS9wxa.js      |  60 ++++
 backend/ui/index.html                    |   4 +-
 backend/ui/ui_version.json               |   6 +-
 backend/worker/__main__.py               |  22 +-
 deploy/cloudrun/Dockerfile               |  22 +-
```

```
 deploy/cloudrun/README.md                |   9 +-
 deploy/gcp/nightly_regression/README.md  | 197 +++++++-----
 docs/COMPUTE_DECOUPLED_SERVING/README.md |   6 +-
 docs/NIGHTLY_ENGINE_REGRESSION.md        |   1 +
 docs/README.md                           |   1 +
 scripts/verify_compute_path.py           | 115 +++++++
 tests/test_api_cloud_dispatch.py         | 528 ++++++++++++++++++++++++++++++++++++
 tests/test_api_endpoints.py              |  87 ++++-
 tests/test_async_jobs.py                 |  61 +++-
 tests/test_compute_backend.py            |  80 +++++
 tests/test_database.py                   |  20 +-
 tests/test_flywheel.py                   | 170 ++++++++++
 tests/test_job_worker_loop.py            |   8 +-
 tests/test_locale_analytics.py           |   6 +-
 tests/test_storage_gdrive_api.py         | 186 +++++++++++
 36 files changed, 2304 insertions(+), 256 deletions(-)
```

43. assistant (2026-06-22T18:45:37.435Z)

Both my files changed on `origin/master` (cloud_run.py: small 3-line change; test file: +80 lines). I must re-apply against the real base, not stash-pop. Let me see exactly what #341 did to `cloud_run.py` so I re-apply cleanly:

44. user (2026-06-22T18:45:39.644Z)

```
=== cloud_run.py change #323..origin/master ===
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 86ff71e..01c2848 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -129,7 +129,8 @@ class CloudRunCompute(ComputeBackend):
         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc)
-        return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
+        return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report,
+                             execution=str(execution or ""))

     # --- storage-backed completion signal (bucket-poll) ---
     def _read_marker(self, done_uri: str) -> Optional[dict]:
=== test file new tests (names) added in origin/master ===
+def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
+def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
+def test_dockerfile_uses_micromamba_not_anaconda_tos():
```

45. assistant (2026-06-22T18:45:57.235Z)

`cloud_run.py`'s #341 change is in `CloudRunCompute.run_infer` (line ~129) ? a different method than mine (`GoogleCloudRunJobClient.run_job`, ~line 64), so no conflict. The test file added 3 unrelated tests, none named like mine. I'll discard my working-tree edits (backed up) and re-apply cleanly on a fresh branch cut from `origin/master`:

46. user (2026-06-22T18:46:00.221Z)

```
branch 'fix/cloud-run-missing-project-guard' set up to track 'origin/master'.
Previous HEAD position was 9b37269 docs(backlog): track golden-regression-fixtures DoD remainder
(M4, P0 rename) + optional O1/O2 (#323)
Switched to a new branch 'fix/cloud-run-missing-project-guard'
=== branch ===
fix/cloud-run-missing-project-guard
=== HEAD (should be #341 d92cc10) ===
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
```

```
skip_ai_handoff forward) (#341)
=== clean tree on tracked files? ===
```

47. assistant (2026-06-22T18:46:07.010Z)

Now on a clean branch at `origin/master` (#341). Let me re-read the current `cloud_run.py` around `run_job` and re-apply my edits against this fresh base:

48. user (2026-06-22T18:46:08.138Z)

```
60      class GoogleCloudRunJobClient(CloudRunJobClient):
61          """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64          def run_job(self, job_name: str, argv: List[str], *, region: str,
65                      project: Optional[str] = None) -> str:
66              try:
67                  from google.cloud import run_v2  # type: ignore[attr-defined]
68              except Exception as exc:  # pragma: no cover - real SDK not present in CI
69                  raise RuntimeError(
70                      "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                      "control plane (it is intentionally NOT a CI/test dependency)."
72                  ) from exc
73              client = run_v2.JobsClient()  # pragma: no cover - exercised only on the real M7
run
74              name = client.job_path(project, region, job_name)
75              overrides = run_v2.RunJobRequest.Overrides(
76
container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77              )
78              op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79              return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
```

49. assistant (2026-06-22T18:46:11.807Z)

The `run_job` method is unchanged by #341. Re-applying my two edits against this base:

50. user (2026-06-22T18:46:15.862Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\compute\cloud_run.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

51. user (2026-06-22T18:46:20.171Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

52. user (2026-06-22T18:46:26.719Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_compute_backend.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

53. assistant (2026-06-22T18:46:36.657Z)

Re-applied cleanly. Now verify on the new base ? ruff + mypy first:

54. user (2026-06-22T18:47:27.523Z)

```
=== ruff touched files ===
All checks passed!
=== mypy backend training (no-incremental, SDK installed) ===
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 138 source files
=== pytest-cov present? ===
pytest-cov 7.1.0
```

55. assistant (2026-06-22T18:47:31.296Z)

ruff clean, mypy `Success` (138 files), pytest-cov present. Now the full suite with coverage ? replicating the exact CI command (floor 70). Let me also capture `cloud_run.py`'s own coverage to prove the improvement:

56. user (2026-06-22T18:49:32.884Z)

Command running in background with ID: bacrfugc8. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output. You will be notified when it completes. To check interim output, use Read on that file path.

57. assistant (2026-06-22T18:49:40.595Z)

Full suite is running in the background. Let me check interim progress:

58. user (2026-06-22T18:49:44.504Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

59. assistant (2026-06-22T18:49:58.571Z)

The suite is running (pytest writes its summary at the end). I'll wait for it to complete before committing ? green local suite is a prerequisite. I'll continue as soon as it finishes.

60. user (2026-06-22T18:49:59.395Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

61. assistant (2026-06-22T18:50:07.263Z)

I'll wait for the completion notification rather than polling.

62. user (2026-06-22T18:50:24.310Z)

```
<task-notification>
<task-id>bacrfugc8</task-id>
<tool-use-id>toolu_01GJJFS9Gvw3tBUvyaznYVhZ</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bacrfugc8.output</output-file>
<status>completed</status>
<summary>Background command "Run full suite with coverage (CI command)" completed (exit code
0)</summary>
</task-notification>
```

63. assistant (2026-06-22T18:50:27.572Z)

Suite completed with exit code 0. Let me read the full output to confirm the result and coverage:

64. user (2026-06-22T18:50:28.837Z)

```
backend\storage\local.py                              49      3    94%
backend\storage\ref.py                                48      0   100%
backend\storage\s3.py                                 47      5    89%
backend\tools\__init__.py                              0      0   100%
backend\tools\export.py                               75      2    97%
backend\tools\rally_slicer.py                        105     47    55%
backend\utils\__init__.py                              0      0   100%
backend\utils\app_home.py                             40      0   100%
backend\utils\ffmpeg.py                               24      0   100%
backend\utils\geometry.py                             29      0   100%
backend\utils\hashing.py                               9      0   100%
backend\utils\logging_setup.py                        11      0   100%
backend\utils\paths.py                                33      3    91%
backend\utils\redact.py                               16      0   100%
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\single_instance.py                      43     11    74%
backend\utils\video.py                                26      3    88%
backend\utils\workspace.py                            49      0   100%
backend\worker\__init__.py                             0      0   100%
backend\worker\__main__.py                           304     29    90%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             129      9    93%
------------------------------------------------------------------------
TOTAL                                              10342   1462    86%
Required test coverage of 70% reached. Total coverage: 85.86%
1488 passed, 7 warnings in 168.88s (0:02:48)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

65. assistant (2026-06-22T18:50:33.957Z)

Full suite: **1488 passed**, total coverage **85.86%** (floor 70 cleared). Let me confirm `cloud_run.py`'s specific coverage (proving the new guard lines are covered):

66. user (2026-06-22T18:50:34.963Z)

(Bash completed with no output)

67. assistant (2026-06-22T18:50:42.136Z)

Let me run focused coverage on `cloud_run.py` to demonstrate the new guard lines are covered:

68. user (2026-06-22T18:50:53.748Z)

```
Name                            Stmts   Miss  Cover   Missing
backend\compute\cloud_run.py       74     11    85%   74-87, 134-136, 162-164
TOTAL                              74     11    85%
15 passed in 8.19s
```

69. assistant (2026-06-22T18:51:20.063Z)

`cloud_run.py` is at **85%** ? the new guard lines are covered (the missing 74-87 are the `# pragma: no cover` real-SDK path), and the compute suite grew to 15 tests. All local gates are green:

- **ruff**: clean
- **mypy backend training** (SDK installed, no-incremental): `Success` ? the latent `arg-type`

```
error is gone
- **full suite**: 1488 passed, **85.86%** coverage (floor 70 cleared)

Now commit (explicit paths, after re-verifying HEAD), push, and open the PR:
```

70. user (2026-06-22T18:51:26.306Z)

```
=== HEAD re-verify (mine) ===
fix/cloud-run-missing-project-guard
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
=== stage explicit paths ===
M  backend/compute/cloud_run.py
M  tests/test_compute_backend.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
[fix/cloud-run-missing-project-guard fc8fb14] fix(compute): guard GoogleCloudRunJobClient
against an unset GOOGLE_CLOUD_PROJECT
 2 files changed, 25 insertions(+)
```

71. assistant (2026-06-22T18:51:31.609Z)

```
Committed `fc8fb14`. Verify the branch contains only my commit vs `origin/master`, then push and
open the PR:
```

72. user (2026-06-22T18:51:35.096Z)

```
=== commits on branch vs origin/master (must be ONLY mine) ===
fc8fb14 fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT
=== push ===
remote: Create a pull request for 'fix/cloud-run-missing-project-guard' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/cloud-run-
missing-project-guard
remote:
branch 'fix/cloud-run-missing-project-guard' set up to track 'origin/fix/cloud-run-missing-
project-guard'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      fix/cloud-run-missing-project-guard -> fix/cloud-run-missing-project-guard
```

73. assistant (2026-06-22T18:51:42.758Z)

```
Branch isolation confirmed (only my commit). Now create the PR:
```

74. user (2026-06-22T18:51:55.197Z)

```
https://github.com/Khelsutra/badminton-highlight-indexer/pull/342
```

75. assistant (2026-06-22T18:52:00.740Z)

```
PR created: [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342). Now let
me watch CI. First, see what checks are registered:
```

76. user (2026-06-22T18:52:12.859Z)

```
Built wheel smoke (install + launch)    pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859922
Dependency license scan (no copyleft in runtime deps)   pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859677
Full test suite (offline, mocked)       pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859930
Golden fixtures cross-OS (macos-latest) pending 0       https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794860679
Golden fixtures cross-OS (windows-latest)       pending 0
```

https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859801
Leak scan (PII / attribution guard) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859809
Syntax sweep + import smoke (no GPU stack) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859736
Ruff lint pass 9s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859797
Golden fixtures cross-OS (ubuntu-latest) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859796
Mypy (lenient baseline) pending 0 https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859772
=== mergeable state ===
{"mergeable":"MERGEABLE","review":"","state":"UNSTABLE"}

77. assistant (2026-06-22T18:52:21.276Z)

CI is running (Ruff already ?). I'll watch the checks in the background ? `gh pr checks --watch` exits when all complete (non-zero if any fail):

78. user (2026-06-22T18:52:22.134Z)

Command running in background with ID: bncmmz9h2. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bncmmz9h2.output. You will be notified when it completes. To check interim output, use Read on that file path.

79. assistant (2026-06-22T18:52:25.479Z)

I'll wait for CI to complete (watching in the background) and then merge once it's verified green.

80. user (2026-06-22T18:54:56.722Z)

<task-notification>
<task-id>bncmmz9h2</task-id>
<tool-use-id>toolu_01Eut5gQqbGptTwtoKYNcbRD</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\bncmmz9h2.output</output-file>
<status>completed</status>
<summary>Background command "Watch PR CI checks until completion" completed (exit code 0)</summary>
</task-notification>

81. assistant (2026-06-22T18:54:59.956Z)

The watch exited 0 (all checks passed). Let me confirm the full check results and mergeability before merging:

82. user (2026-06-22T18:55:03.996Z)

Built wheel smoke (install + launch) pass 1m12s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859922
Dependency license scan (no copyleft in runtime deps) pass 38s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859677
Full test suite (offline, mocked) pass 2m36s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794859930
Golden fixtures cross-OS (macos-latest) pass 28s https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/27976307759/job/82794860679
Golden fixtures cross-OS (ubuntu-latest) pass 21s https://github.com/Khelsutra/badminton-highlight-

```
indexer/actions/runs/27976307759/job/82794859796
Golden fixtures cross-OS (windows-latest)       pass    1m16s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859801
Leak scan (PII / attribution guard)     pass    9s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/27976307759/job/82794859809
Mypy (lenient baseline) pass    29s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859772
Ruff lint       pass    9s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859797
Syntax sweep + import smoke (no GPU stack)      pass    24s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/27976307759/job/82794859736
=== mergeable ===
{"mergeable":"MERGEABLE","state":"CLEAN"}
```

83. assistant (2026-06-22T18:55:12.427Z)

All 10 checks pass; state is **CLEAN**. Merging now (squash, matching the repo's `(#NNN)` convention) and deleting the branch:

84. user (2026-06-22T18:55:19.208Z)

failed to run git: fatal: 'master' is already used by worktree at 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'

```
=== PR state ===
{"mergedAt":"2026-06-22T18:55:18Z","state":"MERGED"}
```

85. assistant (2026-06-22T18:55:28.901Z)

PR **#342 is MERGED** (`mergedAt` set). The `git` error was only gh's post-merge attempt to switch the local checkout to `master`, which is held by another worktree (`.claude/worktrees/tier1-windowing`) ? harmless. Let me confirm the remote branch cleanup and the merge landed on `origin/master`:

86. user (2026-06-22T18:55:34.396Z)

```
 - [deleted]         (none)     -> origin/serving/m11-flywheel-skeleton
 - [deleted]         (none)     -> origin/serving/m12-gdrive-api
   d92cc10..688d28f  master     -> origin/master
=== remote branch gone? (empty = deleted) ===
=== origin/master tip (should be my squash merge #342) ===
688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
d92cc10 feat(serving): make the cloud compute-picker path work + provable (provenance +
skip_ai_handoff forward) (#341)
```

87. assistant (2026-06-22T18:55:43.153Z)

Confirmed merged: `origin/master` advanced to `688d28f fix(compute): guard GoogleCloudRunJobClient... (#342)`, and the remote branch was deleted. Let me tidy the local state ? return to a detached HEAD on the new `origin/master` (matching how the session started, since `master` is held by another worktree), delete the merged local branch, and clear my scratch files:

88. user (2026-06-22T18:55:45.436Z)

```
HEAD is now at 688d28f fix(compute): guard GoogleCloudRunJobClient against an unset
GOOGLE_CLOUD_PROJECT (#342)
Deleted branch fix/cloud-run-missing-project-guard (was fc8fb14).
=== final state ===
688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
```

89. assistant (2026-06-22T18:55:59.605Z)

Done ? shipped and merged. ?

Summary

PR [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342) ? merged to `master` as `688d28f`.

What it fixes
`GoogleCloudRunJobClient.run_job` passed `project` (`Optional[str]`, from `GOOGLE_CLOUD_PROJECT` which can be `None`) straight to `JobsClient.job_path(project: str, ?)`. A guard now validates `project` **before** the lazy SDK import and raises a clear `RuntimeError` naming `GOOGLE_CLOUD_PROJECT` ? which both narrows `Optional[str]` ? `str` for mypy and fails loudly instead of dispatching a bogus `projects/None/?` path.

Verified (not rubber-stamped)
- **Reproduced the latent bug first**: with `google-cloud-run` installed, `mypy` flagged `cloud_run.py:74` (`"str | None"` vs expected `"str"`); confirmed it was a stale-cache false-pass via `--no-incremental`.
- **mypy** `backend training` (SDK installed): `Success, 138 files` ? error gone.
- **ruff**: clean on touched files.
- **Full suite locally**: `1488 passed`, total coverage **85.86%** (floor 70).
- **Coverage ?**: new `test_real_client_rejects_missing_project` (covers `None` and `""`); compute suite 14 ? 15 tests; new guard lines are covered.
- **CI: all 10 checks green** (CLEAN) ? including the full suite, mypy, ruff, cross-OS golden fixtures, wheel smoke, license/leak scans.

Process notes
- The branch base had moved (`origin/master` was at #341, ahead of the session's detached HEAD at #323, and #341 had touched both files). I re-cut the branch from `origin/master` and re-applied cleanly, verified `origin/master..HEAD` contained only my commit, and staged explicit paths ? per the shared-checkout branching-safety convention.
- Squash-merged + remote branch deleted. The one `git` error during merge was only gh trying to switch the local checkout to `master`, which is held by the `tier1-windowing` worktree ? harmless; the merge succeeded. Local checkout returned to a detached HEAD on the new `origin/master`.

This fell under the "merge my own verified-green engineering-fix PRs" working agreement (a type-safety/correctness fix, not product/strategy/licensing), so I merged it. Want me to refresh the session handoff / current-state memory to reflect this landing?

90. user (2026-06-23T09:29:14.548Z)

oh yes please do update the memories, handoffs and docs with the completed work!

91. assistant (2026-06-23T09:29:29.200Z)

Let me read the current memory/handoff files so I update them accurately (not duplicate or overwrite live state):

92. user (2026-06-23T09:29:30.225Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-47 of 189 total (84853 tokens, cap 25000). Call Read with offset=48 limit=47 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
```

Current state ? updated continuously (see [[working-agreement-checkpoints]])

> **Scope of this file = LIVE only:** the current handoff + *today's* checkpoints + open PRs/decisions. Keep it read-first-short (?1 page). Older checkpoints are archived in [[current-state-archive]] ? when today's checkpoints age out (next clean-slate day), move them there, don't let them pile up here.

Checkpoint: 2026-06-23 (**CLOUD TOGGLE LIT + VALIDATION METHOD + LIVE E2E PROVEN + user guide + droplet full-suite**). master `688d28f`+. Owner: "light up the cloud compute toggle with Cloud Run + a good validation method to ensure the intended path was taken" ? then "run the test on the droplet with a full install" ? "write a doc to help users set up + create reels." ALL DONE:

- **? Validation method ? engine [#341](https://github.com/Khelsutra/badminton-highlight-indexer/pull/341) MERGED (`d92cc10`):** compute-path PROVENANCE ? the job result carries `compute={path:"in_process"|"cloud_run"|"not_run", requested, execution, job, region, outputs_uri}` (the cloud `execution` = the real Cloud Run execution name, GCP-verifiable). `ComputeResult.execution` threaded via `run_infer`; honest failure records (path=cloud_run only when it actually dispatched+ran, else not_run); `[COMPUTE]` audit log; **`scripts/verify_compute_path.py`** asserts the path + cross-checks the execution in GCP. **Also forwards `indexing.skip_ai_handoff` to the cloud job** so the picker's cloud path yields rallies without a Gemini key. Adversarial review caught 2 honesty bugs (false "report carries it" claim; misleading log on a never-dispatched reject) ? fixed. Suite 1480?.
- **? LIVE E2E PROVEN ($0 after teardown):** rebuilt the Cloud Run L4 image + Job; ran a local **cloud-serving** engine (`deployment.profile=cloud-serving` + `skip_ai_handoff=true` override + `CLOUD_RUN_JOB/REGION=asia-southeast1/PROJECT` + `storage.output_root`) ? `/api/capabilities` reported **`cloud_available:true`** (toggle lit). `verify_compute_path.py --compute cloud` ? **`path=cloud_run`, execution `rally-worker-qbcqb`** (GCP cross-check: succeededCount=1) ? **22 rallies** (skip_ai_handoff forward works). `--compute local` ? **`path=in_process`** ?. Both directions provably correct. Proof logs in `artifacts/cloudrun_lightup/`. Job+image+AR repo+prefix deleted.
- **? Verify-script Windows bug ? [#344](https://github.com/Khelsutra/badminton-highlight-indexer/pull/344) OPEN:** `--verify-gcp` spawned bare `gcloud` (Windows = `gcloud.cmd`, WinError 2) ? resolve via `shutil.which`. Found during the live run (path assertion passed first).
- **? Sibling merged [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342)** (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` Optional[str] guard I spawned as a chip; that latent mypy nit is fixed.
- **? Droplet FULL-install suite GREEN:** pushed master?`root@168.144.126.176:~/bhi-fulltest`, fresh venv + `pip install -e .[dev,auth,storage-gcs,storage-s3]` + torch CPU + ffmpeg/cv2 libs ? **1461 passed, 12 skipped** (the skips = the private `obsreport` git+ssh dep, not installed). Droplet restored (dir removed, 1.6G freed).
- **? User guide ? [#343](https://github.com/Khelsutra/badminton-highlight-indexer/pull/343) OPEN:** new **`docs/CREATING_REELS.md`** (end-user: prereqs?`indexer --app`?wizard?reel via web UI AND the API chain capabilities?upload/gs://?process[202+poll]?query?compile[202+poll]?download; +cloud toggle +troubleshooting). Researched via a verify-then-write workflow (every command/endpoint/flag checked vs the shipped engine; LIGHT-tier-honest). Indexed in docs/README.md + DOC_STATUS.md §3d.
- **? NEXT:** merge #343+#344 on green; clean worktrees (bhi-computeval/bhi-userdoc/bhi-verifyfix). Owner follow-up to make the toggle live on the ALPHA = deploy a persistent Cloud Run Job + flip the droplet engine to cloud-serving (a deploy decision, not code). [[cloudrun-gpu-serving-gotchas]] [[gcp-vm-always-delete]] [[working-agreement-merging]]

Checkpoint: 2026-06-22 (**ASYNC `/api/compile` [#329] ? engine [#339] + SPA [#108] BUILT; kills the Cloudflare 524 on slow reel stitches**). engine PR [#339](https://github.com/Khelsutra/badminton-highlight-indexer/pull/339) (branch `feat/async-compile`, CI running?merge on green), khelsutra [#108](https://github.com/avidullu/khelsutra/pull/108) **MERGED** (`ed33a51`). The owner's first real reel 524'd (origin timeout) while the reel actually built ? `/api/compile` ran the ffmpeg stitch SYNC (~3min on the 2-core droplet > CF's ~100s edge). **Fix = make it async, mirroring `/api/process`:** submit ? **202 + compile job id**; cheap checks (video/segments/floor/safe-name) fast-fail 4xx at submit; the stitch runs on the worker; the client **polls `/api/jobs/{id}`**. **Engine #339:** schema **v9** (`jobs.kind` 'process'|'compile' NULL?process

+ `jobs.params` JSON; additive/idempotent) + `create_compile_job` + `_run_claimed_job` dispatches on `kind` (compile?`_execute_compile`, else unchanged process path; cost/flywheel process-only) + `_resolve_compile` shared submit/worker + a stitch error ? clean `done`+`result.status=failed` (no crash) + the in-repo `backend/api/static/app.js` polls + **re-bundled `backend/ui/`** from #108. Suite **1473?**, ruff+mypy clean, cov **85.60%**; adversarial review clean (its one finding ? "clients must poll" ? fixed via app.js + the re-bundle). **SPA #108:** `compileReel` submit?`pollCompileJob`?`CompileResult`; a `done`+`failed` result throws an ApiError carrying the engine message (so classifyError surfaces it, not "engine unreachable"); `onBuild` unchanged (still `highlightUrl`, split-origin-safe); 228 web + 95 site tests green. ? **schema head is now v9 = compile job (NOT compute-picker ? that's v8); M9-consent migration ? v10** (the prior checkpoint's "M9-consent=v9" is now stale ? check `PRAGMA user_version`). **Deploy:** #339 ships the API + the poll-aware bundle together ? deploy as ONE unit (don't deploy async engine vs an old sync bundle). [[gotcha-shared-checkout-branch-collisions]] [[working-agreement-merging]]
24
25   **Checkpoint:** 2026-06-22 (**M11 + M12 serving-code DONE ? 2 PRs merged; next = pair on the alpha go-live**). Owner: "build 3 (M11) then 4 (M12) then we pair on the alpha (2); tell me the counsel inputs and I'll have them ready." Both Claude-doable serving items merged on green (isolated worktree off origin/master): **M11** data-flywheel skeleton [#336](https://github.com/Khelsutra/badminton-highlight-indexer/pull/336) (`334ad0f`) ? `backend/flywheel.py` capture/shadow_eval/propose_promotion + `FlywheelConfig` (default-OFF) + gated `_maybe_run_flywheel` hook; **INERT via TWO guards** (`enabled=False` AND `consent_attested` faked-deny ? captures nothing in prod); reuses workspace/eval/promotion/storage; **NO migration**; +14 tests incl. a guardrail-lock; adversarial review (`wf_1ee5ffdf`) inert_guarantee=yes. **M12** `gdrive-api://` OAuth StorageBackend [#338](https://github.com/Khelsutra/badminton-highlight-indexer/pull/338) (`819e395`) ? `backend/storage/gdrive_api.py` (path?Drive-id resolve, **idempotent** put, lazy Google imports, injected-fake hermetic tests) + scheme wiring (ref/`__init__`/models + hyphen?underscore settings); adversarial review (`wf_ae8c9b0f`) no_regression=yes, folded the idempotent-put fix. Suite 1467? cross-OS, ruff+mypy clean. ? **schema ladder drifted mid-session: v7=`locale`, v8=compute-picker ? M9-consent migration is now v9** (check `PRAGMA user_version` first). **? NEXT = OWNER-DRIVEN alpha go-live pairing** (CF tunnel+Pages+Access + Gemini-key rotation per `khelsutra/deploy/DEPLOY_ALPHA.md`); M11-live + M9-consent need the counsel ToS/DPA 7-pt checklist ([[ui-driven-cloud-serving-gap]]). Sibling tracks advanced in parallel: M6?/api/process (#333), Cloud-Run env (#334), the SPA compute-picker + real Cloud-Run-GPU validation (below). [[lesson-verify-engine-surface-before-scoping]]
26
27   **Checkpoint:** 2026-06-22 (**LOCAL-APP ? CLOUD RUN path ? ? ALL 4 ITEMS DONE (5 PRs merged + real-GPU e2e VALIDATED + proof on the local box + $0 teardown)**). engine master `334ad0f`+, khelsutra master `f67dfe7`. Owner: *"build the cloud Run path in the order + ensure due-diligence testing; I approve the GPU VM budget until you get a clean e2e screencast on my local machine."* APPROVED ORDER = (1) API?compute-seam wiring ? (2) P3a micromamba Dockerfile ? (3) SPA compute-picker (engine 3a + SPA 3b) ? (4) real Cloud Run validation + screencast. **PRs: #333 (1), #334 (2), #335 (3a), khelsutra #104 (3b), #337 (the cloud-dep fix item-4 found).**
28   - **? #104 ? compute-picker SPA half (3b)** (khelsutra master `f67dfe7`): IngestPanel probes `/api/capabilities`, shows a "On my computer"/"In the cloud" toggle ONLY when `deployment.cloud_available`, gates CLOUD behind a required cost-ack (disabled controls + early-return), threads `compute` ? `POST /api/process`. Default-OFF (no picker / `compute` omitted when cloud unavailable). New `ingest.compute.*` strings ×6 locales + site mirror (kn/hi/es/da/id AI-drafted ? native review pending). typecheck + 226 web tests + build + coverage + 89 site tests green; adversarial review clean.
29   - **? ITEM 4 DONE ? real Cloud Run L4 GPU e2e VALIDATED, both directions, $0 teardown.** Proof bundle on the local box = **`artifacts/cloudrun_e2e_validation/`** (`RUN_TRANSCRIPT.md` + `cloudrun_e2e_reel.mp4` 22-rally 1080p reel + `contact_sheet.png` + Half-A/B logs + intervals + `TEARDOWN.md`). **Half A** (direct execute) and **Half B** (the LOCAL control plane dispatching `worker infer` in cloud-serving mode ? bucket-poll ? ingest) BOTH produced **22 rallies** on `mahadevpura_smoke_180s.mp4` (deterministic vs the prior local-L4 run), `cuda True`, 325s detect; the local box then stitched the cloud-detected intervals into the reel (the M6 split proven). **Total spend ? $0.60, $0 ongoing** (Job + image + AR repo + validation prefixes all deleted; corpus untouched; worktrees removed).
30   - **? The first real L4 execution CAUGHT A BUG the mocks couldn't ? engine [#337](https://github.com/Khelsutra/badminton-highlight-indexer/pull/337) MERGED** (`5fb55e9`): the Job booted + GCSFuse-mounted the bucket, then died at the first gs:// fetch ? `ModuleNotFoundError: No module named 'google.cloud'` (`backend/storage/gcs.py:24`); the Dockerfile did a bare `pip install .` but the cloud worker does its gs:// I/O through

`backend.storage`. Fix = `pip install ".[storage-gcs]"` + a structural guard (suite 1457?).
31   - **? REUSABLE GOTCHAS for the next cloud run:** (1) Cloud Run **L4 GPU is region-limited** to `asia-southeast1/europe-west1/europe-west4/us-central1/us-east4` (NOT Mumbai) ? use **asia-southeast1 (Singapore)** + **`--no-gpu-zonal-redundancy`** (zonal-redundant GPU is capacity-blocked). (2) Weights (WASB-C3 not baked) ? **GCS volume mount** of the corpus bucket ? `/mnt/corpus`, `WASB_WEIGHTS=/mnt/corpus/models/wasb_badminton_best.pth.tar` (no image change). (3) **Git Bash mangles `/mnt/...` flag values (MSYS path-conv) ? run gcloud GCS-mount/job commands via the PowerShell tool** (`MSYS_NO_PATHCONV=1` breaks gcloud's launcher). (4) cloud-serving sets `skip_ai_handoff=False` ? **must override `--set indexing.skip_ai_handoff=true`** for a pure-engine (no-Gemini-key) run, else the hybrid segmenter `return []` (0 rallies). (5) Cloud Run pins the image DIGEST at create ? `jobs update --image ?:latest` after a rebuild. (6) The local control plane needs `pip install google-cloud-run` + ADC (`gcloud auth application-default`). **?? Owner literal-screencast option:** replay `indexer --app` against a cloud-configured engine + drive the SPA picker while recording. [[gcp-vm-always-delete]] [[compute-stack-gcp-only]] [[ui-driven-cloud-serving-gap]]
32   - **? #333 ? API?Cloud-Run wiring** (COMPUTE_DECOUPLED M6 ? the API path). `/api/process` now offloads DETECT to a Cloud Run GPU Job when `deployment.compute_target=cloud_run` + ingests its `intervals.csv` rallies back into the API DB so query/compile are unchanged. `backend/main.py::_maybe_dispatch_remote` + `_ingest_remote_segments`; **DEFAULT-OFF** (returns None for every non-cloud target ? in-process path byte-identical). Worker gained `_fail(rc)` failure-markers (no more 30-min hang on validation/segmenter fail ? the major review find). Cloud path requires a `gs://`-resolvable input (fast-fail on local) + `storage.output_root`. Destroy-after-confirm preserved (prune partial cloud rows on any failure).
33   - **? #334 ? P3a Dockerfile** (`deploy/cloudrun/Dockerfile`): micromamba + conda-forge (BSD, **no Anaconda ToS**) for the py3.8 WASB env, replacing Miniconda+`conda tos accept`. Cites ADR-005 (permissive licensing).
34   - **? #335 ? compute-picker engine half (3a)** (master `6d3d054`): `ProcessRequest.compute` (`"local"`|`"cloud"`|None) per-request override + `get_compute_backend(target_override=)` + `_cloud_available()` + `deployment.cloud_available` in `/api/capabilities` (SPA shows the toggle only when cloud can dispatch). **Adversarial review (1 Agent) caught a BLOCKER:** `/api/process` is async (202?persist job?worker reconstructs the request) so `compute` was dropped across the boundary ? **schema v8 `jobs.compute` column** + `create_job`/`_job_to_request` thread it through; boundary regression tests lock it. Suite **1443?**, ruff+mypy clean, cov **85.39%**, all 10 CI green.
35   - **? NEXT (the order's tail):** **3b SPA compute-picker** (cross-repo `avidullu/khelsutra`: a "run on my machine"/"run in the cloud" toggle shown when `deployment.cloud_available`, a cost-confirm, send `req.compute`) ? **4 REAL Cloud Run validation** (owner approved GPU spend): `gcloud builds submit -f deploy/cloudrun/Dockerfile` ? `gcloud run jobs create rally-worker --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi` ? control plane `DEPLOYMENT_PROFILE=cloud-serving` + `CLOUD_RUN_JOB/REGION/PROJECT` ? drive a real `gs://` clip end-to-end ? **e2e proof saved to the user's local machine** (the "screencast"). GCP auth = `gcloud` as `avi.dullu@gmail.com` / project `gen-lang-client-0306026826`, corpus bucket `gs://khelsutra-rally-corpus` (videos/labels/weights), GPU quota=1. [[compute-stack-gcp-only]] [[gcp-vm-always-delete]] [[ui-driven-cloud-serving-gap]] [[gotcha-shared-checkout-branch-collisions]]
36
37   **Checkpoint:** 2026-06-22 (**owner-asked green-PR MERGE SWEEP + eval re-ground + backlog; combined master VERIFIED green; 0 open PRs**). master `9b37269`. Owner: "merge the green PRs (coverage + green CI), finish the worker fix + gate-A re-ground, backlog the optional golden-fixtures items, fix/archive completed docs." **Merged 3 ready-green PRs** (no file overlap; none product/strategy/licensing): **#316** DATA_IN_GCS corpus map (docs ? merged FIRST so #319's `§7b` code-comment refs resolve), **#320** nightly Cloud-Run scheduler scripts (shell-only), **#319** worker dated-label fix (a sibling pushed **+2 skip-branch coverage tests** onto the branch ? merged the better version; suite 1418?**1428**). **#322 = gate-A litmus re-ground (this session)** ? confirmed the local "failure" was corpus DRIFT not a regression (golden grew **9?15**, [[golden-corpus-grew-9-to-15]]); re-measured + re-pinned proposal **1.36/0.79/+0.03** & served **?0.023/0.90** + `QUALITY_ITERATIONS.md`; **a parallel session's IDENTICAL #324 was closed as a duplicate** (independent cross-validation, same pins). **#323 = golden-fixtures BACKLOG (this session)** ? `BACKLOG.md` new *Golden regression fixtures* section: §9 DoD remainder (**M4** quality-floor, **P0** name-rename) + optional **O1/O2**. **Docs-archive = NONE archive-ready** (GOLDEN_REGRESSION_FIXTURES non-terminal per its own §9; only candidate = the hardening pointer-stubs, DOC_STATUS finding #4 ? needs owner confirm). **Combined-master VERIFIED: `run_all_tests.py` w/ corpus manifest = 1428?/2 skip (gate-A litmus runs+passes), ruff clean.** ? ~10 concurrent worktrees/sessions ? isolated all work in throwaway worktrees off

`origin/master`, explicit-path staging, cleaned up mine; a sibling's redundant gate-A branch `fix/served-gate-a-regrounded-15clip` is checked out in the MAIN tree (owner trimming concurrency). [[working-agreement-merging]] [[golden-corpus-grew-9-to-15]] [[decision-rigor-norm]]
38
39   **Checkpoint:** 2026-06-22 (**`worker train` dated-label join FIX [#319] + doc [#321] MERGED; eval litmus re-grounded [#324] OPEN for owner**). master `db74f9b`. (1) **[#319]** ? `cmd_train --trajectory-source detector` couldn't match canonical `labels/<YYYY-MM-DD>_<name>.rallies.csv` to `videos/<name>.mp4` (date prefix ? every clip skipped ? `<2 videos` abort; `DATA_IN_GCS §7b#1`). Fix = new helper `_label_clip_name()` in `backend/worker/__main__.py` strips the `.rallies.csv`/`.csv` suffix + a leading `^\d{4}-\d{2}-\d{2}_`; +6 tests (worker cov 82?84%). (2) **[#321]** ? flipped `DATA_IN_GCS §7b#1` to ? RESOLVED + §8 tracker row. Both squash-merged (3-dim adversarial verify workflow: all_ok). (3) **master "broken" locally on `test_served_gate_a_regression`** (`mean_seg_off` 1.16?1.36) ? NOT a code regression: the **golden corpus grew 9?15** (#316 restored the original-6 single-court labels), see [[golden-corpus-grew-9-to-15]]. I re-measured + re-grounded it (PR #324) but a **parallel session had already merged the IDENTICAL fix as [#322]** (`32f4ee6`, same pins 1.36/0.79/+0.03) ? **master already fixed; my #324 closed as a duplicate** (cross-validated ? independent re-measurement, same numbers). Verified `origin/master` litmus = 5 passed locally. master now `9b37269`. [[gotcha-shared-checkout-branch-collisions]] [[decision-rigor-norm]]
40
41   **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? Cloud Run scheduler path FIXED + cloud-validated, [#320] MERGED**). master `48f9d20`. Owner asked Option A (make `register_scheduler.sh` actually work) + a trigger command + email setup. **Two bugs caught by an in-cloud dry-run validation** (build the launcher image ? temp dry-run Cloud Run job ? exec): (1) the launcher container is `COPY . /app` but Cloud Build excludes `.git` ? `launch.sh`'s `git archive` failed ? **fixed**: `launch.sh` now tars the baked tree when no `.git` + takes the SHA from `RALLY_GIT_SHA` (+ a `RALLY_DRY_RUN` smoke mode); (2) `gcloud builds submit --tag` has **no `-f` flag** ? added `cloudbuild.launcher.yaml` + `--config`. Also: `register_scheduler.sh` auto-grants the SA IAM (compute.instanceAdmin.v1+iam.serviceAccountUser+run.invoker) + bakes the SHA + `--run-now`; new **`setup_email.sh`** (Gmail app-pw via STDIN ? Secret Manager + SA secretAccessor). **VALIDATED in-cloud**: image built, the container ran launch.sh (no .git ? tar, SHA=4051ed5), staged the tarball, hit "Skipping VM create" (dry-run) ? with the earlier real-L4 run, the FULL Cloud Scheduler?Cloud Run job?launch.sh?ephemeral-GPU-VM chain is proven. Temp job+image deleted; nightly bucket kept; GCP $0. **? Owner go-live (all from a LOCAL master checkout, NOT the droplet): `create_nightly_bucket.sh` (done) ? upload clips/labels to `gs://?/nightly-inputs/` ? `setup_email.sh` ? `register_scheduler.sh --run-now`. Start on demand: `gcloud run jobs execute nightly-regression-launcher --region asia-south1`.** [[gcp-vm-always-delete]] [[working-agreement-merging]]
42
43   **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ? FAITHFUL WASB run VALIDATED on a real GCP L4 GPU; follow-up [#318]**). Owner: "use GCP GPU VMs (auth/tooling available)." Spun up an **L4** (`rally-nightly-val`, asia-south1-b) via `create_gpu_vm.sh`, set up the real WASB-native conda env + weights (`gs://khelsutra-rally-corpus/models/wasb_badminton_best.pth.tar`), ran `nightly_reelcount.py` with `wasb_hybrid`/`detector_impl=native`/`skip_ai_handoff=true` on `gs://?/videos/mahadevpura_smoke_180s.mp4`: **22 reels, DETERMINISTIC across 3 runs, clean PASS (exit 0), faked regression caught (exit 1, `reel_count 25?22`), cost from real uptime ($0.069/?5.77, 352s @ $0.71/hr)**. **VM DELETED** ($0 idle, no orphan disks ? note `teardown.sh` defaults name `rally-gpu`; set `RALLY_VM_NAME` or `gcloud delete` directly). The real gs://+GPU+WASB run caught **4 deploy bugs the droplet/mocks couldn't**, all fixed in **[#318]** (CI green, merging): (1) `gs://` fetch needs a `dst` (GCS no-passthrough) ? new `_fetch_to` downloads remote into the workdir (+test, also `_load_gts`); (2) `run_on_vm.sh` must export `WASB_REPO_DIR`+`WASB_PYTHON` (not just `WASB_WEIGHTS`); (3) weights default `weights/`?`models/`; (4) manifest `skip_ai_handoff=true` (pure-engine, no paid Gemini ? deterministic+free). GCP auth = `gcloud` as `avi.dullu@gmail.com`/`gen-lang-client-0306026826`, GPU quota=1, weights in `models/`, clips+labels+cached-trajectories in the bucket. Suite 1413?. **? Runner+orchestration now GPU-proven; only the OWNER deploy steps (GCS upload + SMTP secret + `register_scheduler.sh`) remain to go live.** [[gcp-vm-always-delete]] [[compute-stack-gcp-only]] Owner: "use GCP GPU VMs (auth/tooling available)." Spun up an **L4** (`rally-nightly-val`, asia-south1-b) via `create_gpu_vm.sh`, set up the real WASB-native conda env + weights (`gs://khelsutra-rally-corpus/models/wasb_badminton_best.pth.tar`), ran `nightly_reelcount.py` with `wasb_hybrid`/`detector_impl=native`/`skip_ai_handoff=true` on `gs://?/videos/mahadevpura_smoke_180s.mp4`: **22 reels, DETERMINISTIC across 3 runs, clean PASS (exit 0), faked regression caught (exit 1, `reel_count 25?22`), cost from real uptime

($0.069/?5.77, 352s @ $0.71/hr)**. **VM DELETED** ($0 idle, no orphan disks ? note `teardown.sh` defaults name `rally-gpu`; set `RALLY_VM_NAME` or `gcloud delete` directly). The real gs://+GPU+WASB run caught **4 deploy bugs the droplet/mocks couldn't**, all fixed in **[#318]** (CI green, merging): (1) `gs://` fetch needs a `dst` (GCS no-passthrough) ? new `_fetch_to` downloads remote into the workdir (+test, also `_load_gts`); (2) `run_on_vm.sh` must export `WASB_REPO_DIR`+`WASB_PYTHON` (not just `WASB_WEIGHTS`); (3) weights default `weights/`?`models/`; (4) manifest `skip_ai_handoff=true` (pure-engine, no paid Gemini ? deterministic+free). GCP auth = `gcloud` as `avi.dullu@gmail.com`/`gen-lang-client-0306026826`, GPU quota=1, weights in `models/`, clips+labels+cached-trajectories in the bucket. Suite 1413?. **? Runner+orchestration now GPU-proven; only the OWNER deploy steps (GCS upload + SMTP secret + `register_scheduler.sh`) remain to go live.** [[gcp-vm-always-delete]] [[compute-stack-gcp-only]]
44
45   **Checkpoint:** 2026-06-22 (**NIGHTLY E2E ENGINE REGRESSION ? #315 MERGED + DROPLET-VALIDATED**). master `fa2342c`. Owner asked for a *pure-engine* nightly: run the configured+checkpointed pipeline on 2-3 short clips, assert the **reel count is unchanged** + quality + **cost**, **email** the report, on a **GCP GPU VM** (not local/CI ? GitHub CI can't run WASB). Built `scripts/nightly_reelcount.py` (reel-count=`len(play_segments)` EXACT + R2 quality + `backend.billing` cost USD/INR; exit 0/1/2/3/4; re-run-once guard) + `scripts/nightly_email.py` (SMTP via Secret Manager/env, verified-TLS, graceful skip) + `deploy/gcp/nightly_regression/` (self-deleting `run_on_vm.sh` + driver-agnostic `launch.sh` + `register_scheduler.sh` Cloud Scheduler?Cloud Run job + `create_nightly_bucket.sh`) + `eval_baselines/reelcount_manifest.json` (extensible) + tracked doc `docs/NIGHTLY_ENGINE_REGRESSION.md`. **Complements #202** (cached-trajectory CPU re-score) ? this runs the REAL model E2E. **TWO-BUCKET model (owner-agreed):** ephemeral `gs://khelsutra-rally-nightly` (30-day TTL: reports + repo tarball + build staging) vs permanent `gs://khelsutra-rally-corpus` (clips/labels/weights, no TTL) ? a bucket-wide TTL on the dedicated bucket can't touch golden data. Owner asked for a *pure-engine* nightly: run the configured+checkpointed pipeline on 2-3 short clips, assert the **reel count is unchanged** + quality + **cost**, **email** the report, on a **GCP GPU VM** (not local/CI ? GitHub CI can't run WASB). Built `scripts/nightly_reelcount.py` (reel-count=`len(play_segments)` EXACT + R2 quality + `backend.billing` cost USD/INR; exit 0/1/2/3/4; re-run-once guard) + `scripts/nightly_email.py` (SMTP via Secret Manager/env, verified-TLS, graceful skip) + `deploy/gcp/nightly_regression/` (self-deleting `run_on_vm.sh` + driver-agnostic `launch.sh` + `register_scheduler.sh` Cloud Scheduler?Cloud Run job + `create_nightly_bucket.sh`) + `eval_baselines/reelcount_manifest.json` (extensible) + tracked doc `docs/NIGHTLY_ENGINE_REGRESSION.md`. **Complements #202** (cached-trajectory CPU re-score) ? this runs the REAL model E2E. **TWO-BUCKET model (owner-agreed):** ephemeral `gs://khelsutra-rally-nightly` (30-day TTL: reports + repo tarball + build staging) vs permanent `gs://khelsutra-rally-corpus` (clips/labels/weights, no TTL) ? a bucket-wide TTL on the dedicated bucket can't touch golden data.
46   - **RIGOROUSLY TESTED on the DO droplet** (`claude-do-rally-test` 168.144.126.176, CPU/motion segmenter ? validates the runner+alert+cost, not WASB-on-GPU): ran a couple of times clean (deterministic reels=2, exit 0); **faked TWO issues ? alert caught both** ? behaviour regression (`dead_time_merge_gap` 2?5 ? reels 2?1, exit 1) + a missing clip (pipeline error, exit 1, run completes). **The real-machine run caught 2 bugs the mocked tests couldn't** (now fixed + regression-tested): (1) missing `add_video` ? FK fail ? reels silently 0; (2) one bad clip crashed the whole run (exit 2) since `storage.fetch` passes through missing local paths ? now per-clip isolated. Droplet restored to as-found (`~/rally` removed; `/srv/khelsutra-site` untouched). Adversarially reviewed (45-agent workflow, 41?29 findings, HIGH/MED fixed: SMTP cert-verify, re-run-fail?inconclusive, VM self-delete robustness). Suite 1387?, ruff+mypy+leak-scan clean, all 10 CI green.
47   - ? **OWNER to DEPLOY (the nightly code is merged; going live is owner-gated ? needs GCP secrets/quota I lack):** (1) `bash deploy/gcp/nightly_regression/create_nightly_bucket.sh`; (2) upload 2-3 short owned clips+labels to `gs://khelsutra-rally-corpus/nightly-inputs/` + WASB weights to `weights/`; (3) SMTP app-password ? Secret Manager; (4) `launch.sh` dry-run (the FAITHFUL WASB-on-GPU run + the first `--update-baseline` freeze) ? commit `reelcount_baseline.json`; (5) `register_scheduler.sh`. Two open calls (defaulted, easy to change): **driver** (Cloud Run job vs cron vs Claude routine) + **which clips**. Runbook: `deploy/gcp/nightly_regression/README.md`. [[eval-dual-set-regression]] [[compute-stack-gcp-only]] [[gcp-vm-always-delete]]

93. assistant (2026-06-23T09:30:02.258Z)

The `current-state.md` top checkpoint (2026-06-23) already records #342 at line 18 ? a sibling

session noted it (it spawned the original chip; this session implemented + merged it). Let me read the handoff and check whether any doc references this latent bug before deciding what to touch:

94. user (2026-06-23T09:30:03.194Z)

```
1       ---
2       name: session-handoff
3       description: "Living \"you are here\" handoff ? current state, next steps, open
decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key decision, phase
shift, session wrap). \"update the handoff\" forces a full rewrite."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 45ea3a91-654d-4972-9cd3-65f35db54caf
8       ---
9
10      # Session handoff ? badminton-highlight-indexer (rename PENDING) + siblings: sports-
data-collector, rally-annotator, sports-obsreport
11
12      **Last updated:** 2026-06-22 (**M11+M12 serving-code session** ? owner: "build 3 then 4
then we pair on the alpha (2)". **Both Claude-doable engine items DONE + merged:** **M11** data-
flywheel skeleton [#336](https://github.com/Khelsutra/badminton-highlight-indexer/pull/336)
(`334ad0f` ? default-OFF + consent FAKED-DENY, inert; reuses workspace/eval/promotion; NO
migration) + **M12** `gdrive-api://` OAuth StorageBackend
[#338](https://github.com/Khelsutra/badminton-highlight-indexer/pull/338) (`819e395` ?
idempotent put, lazy Google imports, injected-fake tests; live per-user OAuth owner-gated). Both
adversarially reviewed, full-suite+cross-OS green, merged on green, in an isolated worktree. **?
NEXT = pair with owner on the ALPHA GO-LIVE (option 2)** ? owner brings CF dashboard + box +
rotated Gemini key; the counsel ToS/DPA inputs (the 7-pt checklist) gate M11 *live* +
M9-consent. ? schema ladder drifted under me this session: v7=`locale`, v8=compute-picker ? the
future M9-consent migration is **v9** (check `PRAGMA user_version` before authoring). Prior
same-day: **maintenance / green-PR merge-sweep session** ? owner asked to merge the green PRs +
finish loose ends. 6 PRs now on master: worker dated-label fix **#319**, nightly Cloud-Run
scheduler **#320**, DATA_IN_GCS map **#316**, doc **#321**, gate-A litmus re-ground **#322**,
golden-fixtures backlog **#323**. engine `origin/master` = `9b37269`, suite **1428?**, **0 open
engine PRs**; docs-archive had no terminal candidate (owner-gated). Full per-PR detail =
[[current-state]] top. ? ~10 concurrent worktrees/sessions ? owner is trimming concurrency
before new work. The serving/alpha plan below is UNCHANGED. Prior **2026-06-21 ALPHA-SHAREABLE
session** ? Claude's half of "make it UI-driven + alpha-shareable" is **DONE**: 3 PRs merged ?
engine **#294** exposure-safety boot posture + khelsutra **#64** SPA `VITE_API_BASE` + **#65**
deploy kit. **KEY CORRECTION:** B-P2 (SPA ingest screen) was ALREADY shipped (#56) ? alpha-
shareable was a DEPLOY/AUTH task, not an SPA build. Owner chose the **SPLIT** topology (SPA on
CF Pages + `api.*` cloudflared tunnel + CF Access). Remaining = OWNER stand-up via
`khelsutra/deploy/DEPLOY_ALPHA.md`; then engine M11/M12. Full record [[ui-driven-cloud-serving-
gap]]). Prior: HUGE serving session ? M1?M9-auth shipped (#279?#291) + real M7 L4 run. **Repo is
under the `Khelsutra` GitHub org**.
13
14      > **Read order:** this file ? [[current-state]] top (dense per-checkpoint detail) ?
`docs/NEXT_STEPS.md` (the prioritized cross-track plan, refreshed 2026-06-21) ?
`docs/DOC_STATUS.md` (doc lifecycle/hygiene).
15
16      ## ? You are here (2026-06-22)
17      **(2026-06-22 M11+M12)** engine `origin/master` = **`819e395`** (past #336 M11 + #338
M12; also sibling #333 M6?/api/process, #334 Cloud-Run env, #6d3d054 SPA compute-picker) ·
khelsutra `origin/master` = `ba38f15` (past #96 Privacy Policy, #100). **The two Claude-doable
serving-code items are DONE** (M11 flywheel skeleton #336 + M12 gdrive-api #338 ? both default-
OFF/inert, adversarially reviewed, merged on green). **Claude's serving + alpha-shareable code
is now complete.** **?? THE ONLY REMAINING ALPHA STEP IS OWNER-DRIVEN: pair on the go-live**
(cloudflared tunnel + CF Pages + CF Access + Gemini-key rotation per
`khelsutra/deploy/DEPLOY_ALPHA.md`) ? Claude guides, owner drives the CF dashboard/box. M11
going *live* + M9-consent need the counsel ToS/DPA inputs (the 7-pt checklist in [[ui-driven-
cloud-serving-gap]] / this session). Other future engine code: M7 real cloud-serving e2e (owner
GCP spend), per-video workspace P1?P6, the rally-quality track (corpus-gated).
18
```

(2026-06-22 sweep) engine `origin/master` = **`9b37269`** (+ today's #316/#319/#320/#321/#322/#323) · engine **~1428** tests green (combined-master verified w/ corpus manifest) · **0 open engine PRs** · no GCP VMs. This was a cross-track maintenance sweep (worker fix, nightly scheduler, GCS-data doc, gate-A litmus re-ground, golden-fixtures backlog) ? the serving/alpha forward-plan below is unchanged. ? a sibling's redundant gate-A branch `fix/served-gate-a-regrounded-15clip` sits in the MAIN checkout; concurrency being trimmed.

(2026-06-21 prior) engine `origin/master` = `0c0e064`+ (past #292/#293/#294) · khelsutra `origin/master` = `4627830` (past #63/#64/#65) · **no GCP VMs (audited $0)** · engine ~1259 tests green.

?? ALPHA-SHAREABLE SESSION (2026-06-21) ? 3 PRs MERGED; Claude's half of the owner's top priority is DONE:

- **KEY CORRECTION (anti-spiral [[lesson-verify-engine-surface-before-scoping]]):** the handoff said "do the B-P2 SPA ingest screen + M7 Cloud Run *service*" ? but **B-P2 was already shipped** (khelsutra #56, live e2e in `DRY_RUN_CUJ.md` §9) and recon showed alpha-shareable is a **DEPLOY/AUTH** task, not an SPA build. CORS is already env-driven; the engine needed no CORS code.
- **Owner decisions (this session):** (1) pursue alpha-shareable via topology **(i) Tunnel-to-a-box**; (2) sub-topology = **SPLIT** (SPA on CF **Pages** + engine on `api.*` **cloudflared tunnel** + CF Access over both ? engine `HOSTING_PLAN.md` §2), single-origin Caddy kept as the alternative.
- **engine [#294](https://github.com/Khelsutra/badminton-highlight-indexer/pull/294) (`0c0e064`) ? exposure-safety boot posture:** `backend/config/startup.py` fails fast at boot (server-only `include_exposure=True`) when `security.edge_auth_enabled` is ON but half-configured ? `EXPOSED_NO_VIDEO_ROOT` (fatal, jail unset), `EXPOSED_EDGE_AUTH_INCOMPLETE` (fatal, CF team/AUD), `EXPOSED_AUTH_EXTRA_MISSING` (warn). **Default-OFF** (edge_auth off ? no-op; worker never opts in). Aligned `auth.resolve_tenant` whitespace-strip. Adversarial 3-lens review (`wf_35b735a9`) ? no bypass. +16 tests; 1259 green cross-OS.
- **khelsutra [#64](https://github.com/avidullu/khelsutra/pull/64) (`8bbc557`) ? SPA `VITE_API_BASE`:** `apiBase()` env-driven (default same-origin `/api`) + `vite-env.d.ts` + 3 tests; build-smoke proved the override bakes in.
- **khelsutra [#65](https://github.com/avidullu/khelsutra/pull/65) (`4627830`) ? alpha deploy kit:** `deploy/DEPLOY_ALPHA.md` (full split runbook + CORS-preflight gotcha + posture test + deferred notes) + `deploy/cloudflared.config.example.yml` + README/tracker pointers.
- **Discipline:** isolated PRs off `origin/master` (khelsutra in a dedicated **worktree** ? main tree had sibling A8 WIP, since merged as #63), default-OFF, adversarial review, full CI green cross-OS, merged own green PRs, tracker+changelog in-PR. **? REMAINING for alpha = OWNER stand-up only** (tunnel + Pages + CF Access + key rotation per `khelsutra/deploy/DEPLOY_ALPHA.md`). **M1?M9-auth (prior sessions) detail below.**

?? HUGE SERVING SESSION (2026-06-21) ? 6 PRs MERGED + a REAL cloud-GPU run, owner BATCH-AUTHORIZED the default-OFF code path + $100/?10k spend: **#285** (docs §8 D16/D17 + P3 reconcile) · **#286 M5** (`backend/config/startup.py` per-profile fail/warn-fast checks + the **L4 profile-parity** proof; worker stays torch-free) · **#287 M6** (`backend/compute/` ComputeBackend registry ? `CloudRunCompute` dispatch a Cloud Run **Job** + bucket-poll; `deploy/cloudrun/{Dockerfile,README}`; mocked) · **#288 M8** (per-job **cost ledger** + v6 migration + versioned `pricebook` $0 + `/cost` API) · **#290 M9-auth** (Cf-Access JWT verify RS256 + `owner_id` tenant tag; security review found NO bypass; alg=none + HS256-confusion locked) · **#291 ops-agent codify + `docs/CLOUD_GPU_DRYRUN_GUIDE.md`** (`deploy/gcp/create_gpu_vm.sh` always bakes the Ops Agent). All **default-OFF**, isolated, **adversarial-reviewed (a workflow per PR)**, full-suite+cross-OS CI green, tracker+changelog in-PR. **? REAL M7 L4 RUN DONE + VERIFIED:** native WASB on a real L4 ? **22 rallies** on the smoke clip ? GCS out; VM deleted ($0, ~$0.21). ? the no-Gemini cloud run needs **`skip_ai_handoff=true`** (the long-flagged "0 rallies" cold-start fix). A 2nd dry run via `create_gpu_vm.sh` **confirmed Ops-Agent auto-installs** + found the **watermark-font gap** (fixed in #292: `bootstrap.sh` now installs `fonts-dejavu-core`). **M1?M4 (prior session) detail below.**

- **M1 ([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279), `ce22791`):** `deployment.profile`+`compute_target` on `AppConfig` + `backend/config/presets.py` (PROFILE_PRESETS + `suggest_profile`) + loader preset/precedence (defaults<preset<config.json<env via `_deep_merge`; no-profile path = exact old shallow merge) + auto-detect-as-SUGGESTION (D15). Pure/additive, profile='' == today. 3 minor review findings folded.
- **M2 ([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280), `e111add`):** `StorageConfig.input_backend`/`input_root` + `backend/storage/{ref.resolve_input_uri,

__init__.resolve_input_ref/acquire_local_input/input_is_local}` + `main.py` API/CLI wired (acquire local; keep source URI for DB/report). **local = identity passthrough (byte-for-byte unchanged)**; bucket/Drive fetched to scratch (mirrors worker `cmd_infer`). Unknown scheme?clean 400 (was a 500 regression ? caught by review); `local://` accepted on resolved key; hosted-mode rejects non-local; OS-independent anchored-path detection (os.path.isabs differs Win+py3.13). 1 major + 6 minor review findings folded. Suite 1105 green, cov 84.84%.
32   - **M3 ([#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282), `2afd54f`):** configurable output/scratch base ? `backend/utils/paths.output_base(config)` (returns `output.output_dir`, default `"output"`); routed all 6 hardcoded `os.path.join("output", ?)` scratch sites in `trajectory_hybrid`/`yolo_hybrid`/`gemini` through it (create+cleanup same var). **DEFAULT `"output"` ? byte-identical** (golden + cross-OS green); a custom `--output-dir` now actually moves scratch (latent bug fixed). Default-OFF ? no Launch Report. `tests/test_output_base.py` + a yolo call-site capture; clean-bill focused review (yolo-coverage minor folded). Suite 1114 green.
33   - **M4 ([#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283), `cee4baa`):** `DeviceContext` (`backend/pipeline/detectors/device.py`) + native WASB **CPU-fallback** via a new `device="auto"` (cuda-if-available-else-cpu). **Explicit `"cuda"` stays STRICT** (healthcheck still hard-fails without a GPU ? no silent slow-CPU downgrade; "cuda default unchanged"). CUDA probe **cached** (healthcheck?run_predict reuse); `_effective_device()` resolves before argv so `"auto"` NEVER reaches `wasb_infer.py`; fail-fast on a typo'd device. `wasb_infer.py` already supports cpu ? no change there. Device-selection tested offline (mocked subprocess); **real CPU-inference run = owner/GPU-verified.** `tests/test_device_context.py` + native-runner additions; clean-bill focused review (2 nits folded). Suite 1126 green, cov 84.96%.
34   **?? NEXT (priority order):**
35   1. **? ALPHA-SHAREABLE ? Claude's half DONE this session (#294/#64/#65).** Engine is exposure-safe by construction; SPA targets a split origin; the deploy kit + runbook are merged. **? REMAINING = OWNER stand-up** (not Claude-doable): on the box `pip install -e .[auth]` + the safe-exposed `config.json` + `RALLY_CORS_ALLOW_ORIGINS` + rotate `GEMINI_API_KEY` + run `backend.main --server`; CF ? cloudflared tunnel (`api.khelsutra.guru`) + CF Pages (`app.khelsutra.guru`, `VITE_API_BASE`) + ONE CF Access app over both + the OPTIONS-preflight bypass. **Runbook = `khelsutra/deploy/DEPLOY_ALPHA.md`.** *(A later UI polish, not the alpha blocker: a compute PICKER in the UI, D13/D15.)* Full record: [[ui-driven-cloud-serving-gap]].
36   2. **M11** (data-flywheel plumbing) ? ? privacy-sensitive (consent/data), give it FRESH focus: `backend/flywheel.py` capture?store?shadow-eval?goldenize reusing `workspace.py::for_video` + `backend/eval/experiment.compare` + `backend/eval/promotion.py::promote_if_served_safe`, behind `flywheel.*`=OFF + a faked `consent_attested`. ? depends on M9's not-yet-landed **v7 consent migration** (M8 took v6) + counsel ToS/DPA; only a default-OFF skeleton is Claude-doable.
37   3. **M12** (`gdrive-api` OAuth StorageBackend) ? `backend/storage/gdrive_api.py` `GDriveApiStorage(StorageBackend)` + a `gdrive-api://` scheme (regex allows the hyphen; add to `_KNOWN`+`_backend_class`+the StorageConfig Literal) vs an INJECTED fake Drive (mirror `gcs.py` DI); keep mount-only `gdrive://`. ? M9-consent cols + per-video P1 = a new **v7** migration (M8 took v6).
38   **? owner-only residual:** real M5 truly-local runlog (template pre-staged) · GCP spend for the M7 *service* deploy + a real e2e · counsel ToS/DPA (M9-consent) · real GCP prices+FX + the M8 usage-emission follow-up · Google OAuth app (M12 real) · CF team-domain/AUD+ingress-lock · repo rename · DO droplet/token retire.
39
40   ## ? Icebreaker for the NEXT session (copy-paste)
41   > Continue the badminton-highlight-indexer ENGINE serving + UX track. FIRST: `git pull --ff-only`, then read memory `session-handoff.md` + `current-state.md` (top) + [[ui-driven-cloud-serving-gap]] + [[compute-stack-gcp-only]].
42   > STATE: **The cloud alpha is LIVE + proven end-to-end** (2026-06-22 night: a real match ? **27 rallies ? reel downloaded**, on the droplet `claude-do-rally-test`/`168.144.126.176` behind CF Access at `app.khelsutra.guru`; fixed live: `app.` DNS?the tunnel, the Gemini key, `RALLY_ALLOWED_HOSTS`). **Engine #294 + M11 #336 + M12 #338, khelsutra #64/#65 all merged.**
43   > **?? TOP NEXT ACTION ? owner LOCKED IN the GCP L4 (2026-06-22 night): spin up the L4 for the NON-GEMINI phase** (native WASB inference + owned-model training, NO Gemini). Run **`bash deploy/gcp/create_gpu_vm.sh`** (defaults `RALLY_GPU=l4` ? `g2-standard-4` 24GB, zone-sweeps Mumbai?Singapore, bakes Ops-Agent) ? pull corpus from `gs://khelsutra-rally-corpus` ([[gcs-data-source-of-truth]]) ? `python -m backend.worker infer|train` with `skip_ai_handoff=true` ? **`bash deploy/gcp/teardown.sh delete` AFTER** ([[gcp-vm-always-delete]]), **$100/?10k cap**. On-demand ~$0.74/hr Mumbai (?$1?2/interactive session, fine); `RALLY_GPU=t4` fallback if L4 stocks out; add a Spot toggle (`--provisioning-model=SPOT`,

currently STANDARD at `create_gpu_vm.sh:41`) before any LONG training run. Full config/pricing = [[compute-stack-gcp-only]].
44 > **Also queued ? 3 alpha-hardening issues from the live run (Claude-doable, default-safe):** khelsutra#111 (SPA survive-refresh + live progress ? the 10-min poll ceiling timed out ~secs before a long job finished), khelsutra#112 (WhatsApp share the VIDEO via Web Share API, not just text), engine#340 (idempotent `/api/process` ? no Gemini RE-BILL on refresh/"Try again").
45 > Discipline: ONE isolated PR per item off `origin/master`, default-OFF + golden/cross-OS-gated, adversarial review before commit, full `run_all_tests.py`+ruff+mypy green, merge own green PRs, tracker+changelog in-PR, checkpoint memory. ? **shared checkout** (engine AND khelsutra ? sibling sessions commit concurrently; khelsutra needs a dedicated **worktree**: junction `web/node_modules` then **`rmdir` the junction BEFORE `git worktree remove`**). ? **the LIVE alpha runs on the droplet** (SSH `~/.ssh/khelsutra_droplet` root) ? don't disrupt the `khelsutra-engine`/`cloudflared` services or the marketing site.
46
47 Prior session delivered the **engine compute/serving + doc-hygiene** track (10 PRs merged: #263?#267, #270, #272?#274, #276):
48 - **M2 FINALIZED on a real GPU** ? `worker train --trajectory-source detector` on a GCP **L4 (Singapore; Mumbai was stocked-out)** ? Gen-0 bundle `gs://khelsutra-rally-corpus/weights/0.1.0-l4/` (honest **n=2 plumbing** result, not a quality model). Both VMs **deleted**.
49 - **DigitalOcean DROPPED for compute ? GCP is the sole stack** (ADR-013; DO GPU not enabled/credit-funded, Paperspace subscription-gated). [[compute-stack-gcp-only]] · [[gcp-vm-always-delete]] · [[gcp-ops-agent-enable]].
50 - **DECOUPLED_COMPUTE delivered + ARCHIVED** ? `docs/archives/past_projects/DECOUPLED_COMPUTE/`.
51 - **NEW tracked subproject: `docs/COMPUTE_DECOUPLED_SERVING/` + ADR-014** ? one pure core (DETECT?WINDOW?STITCH, never branches on profile) + `ComputeBackend`/`StorageBackend` profiles (**truly-local**, **cloud-serving** [Cloud Run GPU runs detect AND stitch, metered], **cpu-mock**). **? AWAITING OWNER M0 DESIGN SIGN-OFF before any code.**
52 - **Doc governance:** `docs/DOC_STATUS.md` register + **archival due-diligence** (honestly-update?then-archive; codified in CLAUDE.md; [[doc-status-register]]); per-video docs reconciled; **data layer** carved into `docs/data_pipeline/` (no-ML), field-ops/PMF?`archives/field-pmf/`, Roora?`archives/roora-vendor/`.
53
54 ? **Shared checkout** ? sibling sessions commit + flip branches here mid-session (hit twice this session). **Branch from `origin/master` explicitly, re-verify HEAD before commit, stage explicit paths, never `git add -A`.** [[gotcha-shared-checkout-branch-collisions]]. ? **`sed -i` over a glob churns CRLF on all matched files** ? separate real changes with `git diff --ignore-cr-at-eol` and revert the noise.
55
56 ## ? Next steps (resume here)
57 1. **? M0?M4 SHIPPED (#279/#280/#282/#283) ? the config/input/output/device SEAM LAYER is COMPLETE ? NOW M5+ (mostly owner-gated).** Decisions locked in README **§2 D7?D15** (D7 scale-to-zero · D8 versioned DB pricebook USD/INR · D9 Cloudflare Access · D10 data-for-reels trade adults-only · D11 Gemini-ON-until-floor + D12/M11 flywheel · D13 user-switchable profiles · D14 NORTHSTAR gdrive-api OAuth = M12 · D15 suggest?confirm). **Engine build sequence:** ~~M1 deployment.profile~~ ? ~~M2 input_backend~~ ? ~~M3 output-base~~ ? ~~M4 DeviceContext + CPU-fallback~~ **ALL DONE.** **? M5** = truly-local profile e2e (preset + startup checks + first committed runlog) ? **? owner real-GPU** (needs a GPU box / owner's WSL machine to run detect+stitch locally, then commit the runlog). **M6/M7** Cloud Run dispatch + container image (? owner GCP spend), **M8** cost ledger + pricebook, **M9** edge-auth (Cf-Access), **M11** data-flywheel harness, **M12** gdrive-api OAuth. **Per-video-workspace P3 (`output/chunks_*`) is now subsumed by M3 ? reconcile/close it.** **Claude-doable WITHOUT owner gating:** the M5 preset+startup-check code (the actual run is owner's); the M8 cost-ledger DB migration + pricebook + aggregation (pure code, owner does the real-cost calibration); the M11 harness plumbing. M6/M7/M9/M12 need owner cloud/counsel sign-off.
58 2. **Per-video workspace P1?P6** (#264 shipped P0): P1 v6 DB migration `workspace_uri` ? P2 worker ? P3 `output/chunks_*` de-hardcode ? ? (Claude-doable, default-OFF).
59 3. **Ops-Agent codification into `deploy/gcp`** (auto-enable observability on every VM) ? small, [[gcp-ops-agent-enable]].
60 4. **[owner]** Grow the golden corpus (highest-leverage) · set ship-gate target #172 · full-corpus training (n?5) for real weights.
61 5. Full prioritized P0/P1/P2 list = `docs/NEXT_STEPS.md` top.
62

```
63      ## ? Open decisions / owner gates
64      - **M0 design sign-off** (the blocker above) + the COMPUTE_DECOUPLED_SERVING §8
questions.
65      - Doc-hygiene leftovers (`DOC_STATUS.md` §4): confirm the 3 COMPLETED top-level pointer-
stubs; `HOSTING_PLAN`/`UI_PROPOSAL` currency.
66      - Housekeeping: retire the shared DO droplet `claude-do-rally-test` (~$24/mo) + the DO
token; rename repo off "badminton".
67
68      ## ? Ramp-up kit
69      - **Memory:** [[current-state]] (read first) · [[compute-stack-gcp-only]] · [[gcp-vm-
always-delete]] · [[gcp-ops-agent-enable]] · [[doc-status-register]] · [[gotcha-shared-checkout-
branch-collisions]].
70      - **Docs:**
`docs/COMPUTE_DECOUPLED_SERVING/{README,TESTING_STRATEGY,architecture.svg,runlogs/}` ·
ADR-012/013/**014** in `docs/archives/decisions/DECISIONS.md` · `docs/NEXT_STEPS.md` ·
`docs/DOC_STATUS.md` · `docs/PER_VIDEO_WORKSPACE.md` · `deploy/gcp/README.md` (GPU runbook;
teardown defaults to `delete`).
71      - **Code anchors (the pure core the serving project builds on):**
`backend/pipeline/detectors/base.py` (DetectorRunner CSV contract) ·
`tracknet_runner.py::trajectory_to_action_windows` ·
`backend/pipeline/stitcher/compiler.py::compile_highlights` ·
`backend/pipeline/segmenters/trajectory_hybrid.py::_resolve_runner` (the compute seam) ·
`backend/storage/*` · `backend/worker/__main__.py` · `backend/utils/workspace.py`.
72
73      ## ? Parallel track (NOT this session ? sibling sessions)
74      **KhelSutra brand-site + product UX/Hindi-Kannada localization** (repo
`Khelsutra/khelsutra` + #275 here) ? [[next-session-ux-localization]] / [[khelsutra-domain-
launch]]. The **rally-quality / owned-model** track is live too (R-series,
`RALLY_QUALITY_RESEARCH.md`; gated on corpus growth). These are background to the
compute/serving resume above.
75
```

95. assistant (2026-06-23T09:30:29.623Z)

The handoff is at 2026-06-22; `current-state` already records #342. Let me check whether any doc references `GOOGLE_CLOUD_PROJECT` / the cloud-run env setup, so a doc note would land somewhere natural rather than being aspirational noise:

96. user (2026-06-23T09:30:30.594Z)

```
Found 8 files
tests\test_compute_backend.py
backend\compute\cloud_run.py
tests\test_api_cloud_dispatch.py
deploy\cloudrun\README.md
backend\main.py
backend\compute\__init__.py
artifacts\cloudrun_e2e_validation\RUN_TRANSCRIPT.md
backend\config\startup.py
```

97. assistant (2026-06-23T09:30:34.812Z)

`deploy/cloudrun/README.md` is the natural home for an env-setup note. Let me see its `GOOGLE_CLOUD_PROJECT` context:

98. user (2026-06-23T09:30:36.072Z)

```
9-> cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does **not**
build this image.
10-
11-## 0. Prerequisites (owner)
12:- A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13-- GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else Singapore
```

```
? see
14-  `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15-- The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16-
17-## 1. Build + push the image (Artifact Registry)
18-```bash
19:PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20-gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21-gcloud builds submit --tag $REGION-docker.pkg.dev\$PROJECT\$REPO\$IMG:latest -f
deploy/cloudrun/Dockerfile .
22-```
--
37-`compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute` from
env:
38-```bash
39-export DEPLOYMENT_PROFILE=cloud-serving
40:export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION  GOOGLE_CLOUD_PROJECT=$PROJECT
41-# A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job instead
of
42-# running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43-python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
```

99. user (2026-06-23T09:30:40.649Z)

```
34
35      ## 3. Wire the dispatcher (control plane)
36      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38      ```bash
39      export DEPLOYMENT_PROFILE=cloud-serving
40      export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44      ```
45
46      ## 4. M7 acceptance (the owner runlog)
47      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52      ## Gotchas
53      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
```

100. assistant (2026-06-23T09:31:29.102Z)

`deploy/cloudrun/README.md` is the natural operator home ? its "Gotchas" section should note the

new fail-fast. I'll do all three: enrich the `current-state` #342 line, add a handoff note, and ship a small doc PR for the README gotcha.

First, the memory updates:

101. user (2026-06-23T09:31:36.516Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

102. user (2026-06-23T09:31:44.684Z)

14 **Checkpoint:** 2026-06-23 (**CLOUD TOGGLE LIT + VALIDATION METHOD + LIVE E2E PROVEN + user guide + droplet full-suite**). master `688d28f`+. Owner: "light up the cloud compute toggle with Cloud Run + a good validation method to ensure the intended path was taken" ? then "run the test on the droplet with a full install" ? "write a doc to help users set up + create reels." ALL DONE:
15 - **? Validation method ? engine [#341](https://github.com/Khelsutra/badminton-highlight-indexer/pull/341) MERGED (`d92cc10`):** compute-path PROVENANCE ? the job result carries `compute={path:"in_process"|"cloud_run"|"not_run", requested, execution, job, region, outputs_uri}` (the cloud `execution` = the real Cloud Run execution name, GCP-verifiable). `ComputeResult.execution` threaded via `run_infer`; honest failure records (path=cloud_run only when it actually dispatched+ran, else not_run); `[COMPUTE]` audit log; **`scripts/verify_compute_path.py`** asserts the path + cross-checks the execution in GCP. **Also forwards `indexing.skip_ai_handoff` to the cloud job** so the picker's cloud path yields rallies without a Gemini key. Adversarial review caught 2 honesty bugs (false "report carries it" claim; misleading log on a never-dispatched reject) ? fixed. Suite 1480?.
16 - **? LIVE E2E PROVEN ($0 after teardown):** rebuilt the Cloud Run L4 image + Job; ran a local **cloud-serving** engine (`deployment.profile=cloud-serving` + `skip_ai_handoff=true` override + `CLOUD_RUN_JOB/REGION=asia-southeast1/PROJECT` + `storage.output_root`) ? `/api/capabilities` reported **`cloud_available:true`** (toggle lit). `verify_compute_path.py --compute cloud` ? **`path=cloud_run`, execution `rally-worker-qbcqb`** (GCP cross-check: succeededCount=1) ? **22 rallies** (skip_ai_handoff forward works). `--compute local` ? **`path=in_process`** ?. Both directions provably correct. Proof logs in `artifacts/cloudrun_lightup/`. Job+image+AR repo+prefix deleted.
17 - **? Verify-script Windows bug ? [#344](https://github.com/Khelsutra/badminton-highlight-indexer/pull/344) OPEN:** `--verify-gcp` spawned bare `gcloud` (Windows = `gcloud.cmd`, WinError 2) ? resolve via `shutil.which`. Found during the live run (path assertion passed first).
18 - **? Sibling merged [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342)** (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` Optional[str] guard I spawned as a chip; that latent mypy nit is fixed.
19 - **? Droplet FULL-install suite GREEN:** pushed master?`root@168.144.126.176:~/bhi-fulltest`, fresh venv + `pip install -e .[dev,auth,storage-gcs,storage-s3]` + torch CPU + ffmpeg/cv2 libs ? **1461 passed, 12 skipped** (the skips = the private `obsreport` git+ssh dep, not installed). Droplet restored (dir removed, 1.6G freed).
20 - **? User guide ? [#343](https://github.com/Khelsutra/badminton-highlight-indexer/pull/343) OPEN:** new **`docs/CREATING_REELS.md`** (end-user: prereqs?`indexer --app`?wizard?reel via web UI AND the API chain capabilities?upload/gs://?process[202+poll]?query?compile[202+poll]?download; +cloud toggle +troubleshooting). Researched via a verify-then-write workflow (every command/endpoint/flag checked vs the shipped engine; LIGHT-tier-honest). Indexed in docs/README.md + DOC_STATUS.md §3d.
21 - **? NEXT:** merge #343+#344 on green; clean worktrees (bhi-computeval/bhi-userdoc/bhi-verifyfix). Owner follow-up to make the toggle live on the ALPHA = deploy a persistent Cloud Run Job + flip the droplet engine to cloud-serving (a deploy decision, not code). [[cloudrun-gpu-serving-gotchas]] [[gcp-vm-always-delete]] [[working-agreement-merging]]
22

103. user (2026-06-23T09:31:59.001Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

104. assistant (2026-06-23T09:32:02.241Z)

Now add a concise note to the handoff's "You are here":

105. user (2026-06-23T09:32:07.796Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

106. assistant (2026-06-23T09:32:13.977Z)

Memory + handoff updated. Now the doc PR. Let me fetch (siblings may have moved master), branch from `origin/master`, and add the operator gotcha:

107. user (2026-06-23T09:32:18.941Z)

```
From https://github.com/Khelsutra/badminton-highlight-indexer
   688d28f..239c8eb  master     -> origin/master
=== origin/master ===
239c8eb fix(scripts): resolve gcloud via PATH in verify_compute_path so --verify-gcp works on
Windows (#344)
Previous HEAD position was 688d28f fix(compute): guard GoogleCloudRunJobClient against an unset
GOOGLE_CLOUD_PROJECT (#342)
Switched to a new branch 'docs/cloudrun-require-project'
docs/cloudrun-require-project
```

108. assistant (2026-06-23T09:32:28.624Z)

Branched from the latest `origin/master` (`239c8eb`, #344 just merged by a sibling). Let me read the current Gotchas section on this base and add the operator note:

109. user (2026-06-23T09:32:29.378Z)

```
52      ## Gotchas
53      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57      - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
58        real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
59        video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
60        (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
61        surfaces it as a `cloud dispatch error` job failure (not a traceback).
62      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
63
```

110. user (2026-06-23T09:32:35.960Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

111. assistant (2026-06-23T09:32:39.986Z)

Doc note added. Now commit (explicit path), push, and open the PR:

112. user (2026-06-23T09:32:45.055Z)

```
=== HEAD re-verify ===
docs/cloudrun-require-project
239c8eb fix(scripts): resolve gcloud via PATH in verify_compute_path so --verify-gcp works on
Windows (#344)
=== stage explicit path ===
M  deploy/cloudrun/README.md
[docs/cloudrun-require-project e5aebb0] docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required
(run_job fails fast if unset)
 1 file changed, 5 insertions(+)
=== only my commit vs origin/master? ===
e5aebb0 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)
```

113. user (2026-06-23T09:32:57.605Z)

```
remote:
branch 'docs/cloudrun-require-project' set up to track 'origin/docs/cloudrun-require-project'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
 * [new branch]      docs/cloudrun-require-project -> docs/cloudrun-require-project
https://github.com/Khelsutra/badminton-highlight-indexer/pull/345
```

114. assistant (2026-06-23T09:33:00.894Z)

```
Doc PR [#345](https://github.com/Khelsutra/badminton-highlight-indexer/pull/345) created.
Watching CI to green, then I'll merge:
```

115. user (2026-06-23T09:33:02.147Z)

```
Command running in background with ID: b4zkvhk63. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\b4zkvhk63.output. You will be notified when
it completes. To check interim output, use Read on that file path.
```

116. assistant (2026-06-23T09:33:08.870Z)

```
I'll wait for CI on the doc PR, then merge it.
```

117. user (2026-06-23T09:36:09.065Z)

```
<task-notification>
<task-id>b4zkvhk63</task-id>
<tool-use-id>toolu_01JwP12afBH4YYiLTTqdahKb</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\b42b585b-e38f-48ef-8573-5cfc6da69801\tasks\b4zkvhk63.output</output-file>
<status>completed</status>
<summary>Background command "Watch doc PR CI to completion" completed (exit code 0)</summary>
</task-notification>
```

118. assistant (2026-06-23T09:36:13.541Z)

```
CI passed (exit 0). Confirming all checks green and mergeable, then merging:
```

119. user (2026-06-23T09:36:17.934Z)

```
Built wheel smoke (install + launch)    pass    1m11s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474368
Dependency license scan (no copyleft in runtime deps)   pass    32s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474435
Full test suite (offline, mocked)       pass    2m38s   https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474245
Golden fixtures cross-OS (macos-latest) pass    20s     https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474455
```

```
Golden fixtures cross-OS (ubuntu-latest)        pass    22s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474417
Golden fixtures cross-OS (windows-latest)       pass    51s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474308
Leak scan (PII / attribution guard)     pass    6s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28016635693/job/82922474579
Mypy (lenient baseline) pass    37s     https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474256
Ruff lint       pass    8s      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474310
Syntax sweep + import smoke (no GPU stack)      pass    23s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28016635693/job/82922474268
=== mergeable ===
{"mergeable":"MERGEABLE","state":"CLEAN"}
```

120. assistant (2026-06-23T09:36:20.638Z)

All 10 checks pass, state CLEAN. Merging and cleaning up:

121. user (2026-06-23T09:36:28.731Z)

```
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'

=== PR state ===
MERGED
 - [deleted]         (none)     -> origin/fix/verify-gcloud-windows
   239c8eb..9329473  master     -> origin/master
=== origin/master tip ===
9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)
(#345)
```

122. assistant (2026-06-23T09:36:38.886Z)

PR **#345 MERGED** to `master` as `9329473`. The git error was again only gh's local-checkout switch (master held by the worktree) ? harmless. Let me clean up git state, then add the #345 citation to the memory notes:

123. user (2026-06-23T09:36:41.400Z)

```
HEAD is now at 9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast
if unset) (#345)
Deleted branch docs/cloudrun-require-project (was e5aebb0).
=== final HEAD ===
9329473 docs(cloudrun): note GOOGLE_CLOUD_PROJECT is required (run_job fails fast if unset)
(#345)
=== clean tracked tree? ===
```

124. assistant (2026-06-23T09:36:43.773Z)

Now cite #345 in the memory notes so the record is precise:

125. user (2026-06-23T09:36:47.436Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\session-handoff.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

126. user (2026-06-23T09:36:53.293Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
```

indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

127. assistant (2026-06-23T09:37:09.568Z)

All updates are landed. Here's the final state.

Memories, handoff & docs ? updated to reflect the completed work

Two PRs merged (both squash-merged, branched from `origin/master`, verified-green, per the merge working agreement):
- [#342](https://github.com/Khelsutra/badminton-highlight-indexer/pull/342) (`688d28f`) ? the `GOOGLE_CLOUD_PROJECT` guard + test (merged earlier this turn)
- [#345](https://github.com/Khelsutra/badminton-highlight-indexer/pull/345) (`9329473`) ? the operator doc note (just merged)

Memory (`~/.claude/.../memory/`):
- [current-state.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md) ? replaced the sibling's brief "#342 chip" stub with the full record: the latent bug, the fix mechanism, the test, suite/coverage numbers, and the stale-`.mypy_cache` gotcha; added the #345 follow-up.
- [session-handoff.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\session-handoff.md) ? added a `(2026-06-23 #342)` "You are here" entry citing both PRs.

Docs (in-repo, shipped via #345):
- `deploy/cloudrun/README.md` ? a "Gotchas" bullet documenting that `GOOGLE_CLOUD_PROJECT` is required and `run_job` now fails fast if unset, so an operator who hits the error knows where to set it.

Notes on scope
- **No new dedicated memory file**: a merged bug-fix is git history, which the memory-hygiene rules say not to duplicate ? the checkpoint pointers in current-state/handoff are the right record.
- **`MEMORY.md` index unchanged**: no new memory files, so no new index line needed.
- **Concurrency-safe**: master moved twice under me during this (siblings merged #344, then my #345); I re-cut from `origin/master` each time, verified `origin/master..HEAD` contained only my commit, and staged explicit paths. Final `origin/master` = `9329473`. Local tree clean; merged branches deleted; scratch files removed.

Both fixes are live on `master` and the session record is current.